

INSTITUTO INFNET
DESENVOLVIMENTO SEGURO DE APLICAÇÕES WEB

ASSESSMENT

MATHEUS FELIPE ALVES SANTOS

RIO DE JANEIRO - 03/10/2026

Repositório com código-fonte:

<https://github.com/MatheusSantos285/clinica-api-devseguro>

Link para o Vídeo de Apresentação:

<https://www.youtube.com/watch?v=eiswub7WgGk>

Link para o Drive com o DFD, o Vídeo e o Arquivo ZIP:

<https://drive.google.com/drive/folders/1yhskhqdlPVdhcm7lJul5iOVTHxm7Mq3J?usp=sharing>

Resumo da Implementação - Exercício 1

Enunciado:

Configure um ambiente Python isolado para o projeto e estruture a aplicação FastAPI inicial, já organizada nos módulos routes, models e database, com pelo menos um recurso RESTful completo (consultas) implementado via APIRouter. Escreva também um teste automatizado com pytest cobrindo pelo menos um caminho de sucesso do endpoint de consultas implementado, criando desde já o primeiro arquivo de testes automatizados que será expandido ao longo do Assessment.

1. Objetivo

O Exercício 1 teve como foco o isolamento de dependências, a estruturação modular do projeto e, principalmente, a implementação de controles preventivos de segurança na borda da aplicação (Security Gates) utilizando recursos nativos do framework e da biblioteca Pydantic.

2. Arquitetura e Modularização

Para garantir que o código escale de forma limpa e facilite a manutenção, as responsabilidades foram divididas em módulos isolados, evitando a concentração de lógica no arquivo principal (main.py). O projeto foi estruturado da seguinte forma:

- **models/**: Concentra os modelos Pydantic, atuando como a camada de validação e filtro de entrada/saída de dados da API.
- **routes/**: Concentra os controladores (APIRouter) e a definição dos endpoints RESTful, sem regras de negócio complexas acopladas.
- **database/**: Módulo dedicado à camada de persistência. Nesta etapa, foi implementado um mock em memória (dicionário) para simular o banco de dados.
- **tests/**: Camada de testes automatizados utilizando pytest e TestClient.

3. Controles de Segurança Aplicados

A construção da API seguiu o princípio de que a segurança real deve ser implementada obrigatoriamente no Back-end, assumindo que o ambiente do cliente é hostil. Os seguintes controles foram aplicados:

3.1. Ingress Filtering e Mitigação de Mass Assignment

Foi criado o modelo de entrada *ConsultaCreate* rigorosamente tipado.

- **Controle:** O Pydantic valida os dados antes de atingirem o controlador, utilizando o parâmetro `Field(..., min_length, max_length)` para delimitar o tamanho das strings.
- **Mitigação:** Isso previne ataques de injeção e estouro de buffer logo na borda. Além disso, mitiga vulnerabilidades de *Mass Assignment* (Injeção de Propriedades), pois qualquer campo extra enviado no corpo da requisição JSON (como tentativas de forçar um id ou alterar um status restrito) é silenciosamente ignorado e descartado.
- **Evidência:** Requisições com dados faltantes ou curtos retornam instantaneamente o erro estruturado 422 Unprocessable Content.

3.2. Egress Filtering e Prevenção de Vazamento de Dados

Foi implementado o modelo de saída *ConsultaResponse* vinculado ao parâmetro *response_model* dos endpoints.

- **Controle:** O FastAPI intercepta o retorno da função e o filtra estruturalmente antes de serializar o JSON final.
- **Mitigação:** Evita a fraqueza conhecida como **CWE-200** (Exposição de Informação Sensível para Ator Não Autorizado). No projeto, a simulação do banco possui o campo sensível `anotacoes_internas`. A adoção do modelo de saída garantiu que esse dado interno nunca fosse vazado publicamente, frustrando a tática de *Discovery* do MITRE ATT&CK.

3.3. Prevenção contra Information Disclosure (Tratamento de Exceções)

No endpoint de busca por ID (GET `/consultas/{id}`), o tratamento de dados não encontrados foi codificado defensivamente.

- **Controle:** Em vez de retornar um mascarado 200 OK vazio, ou permitir que o servidor estoure um stack trace, a API levanta manualmente um `HTTPException`.
- **Mitigação:** O retorno exato de 404 Not Found (e não um 403 ou 500) evita a enumeração de recursos por atacantes (Information Disclosure) e informa corretamente ao cliente e a possíveis WAFs sobre a ausência do recurso.

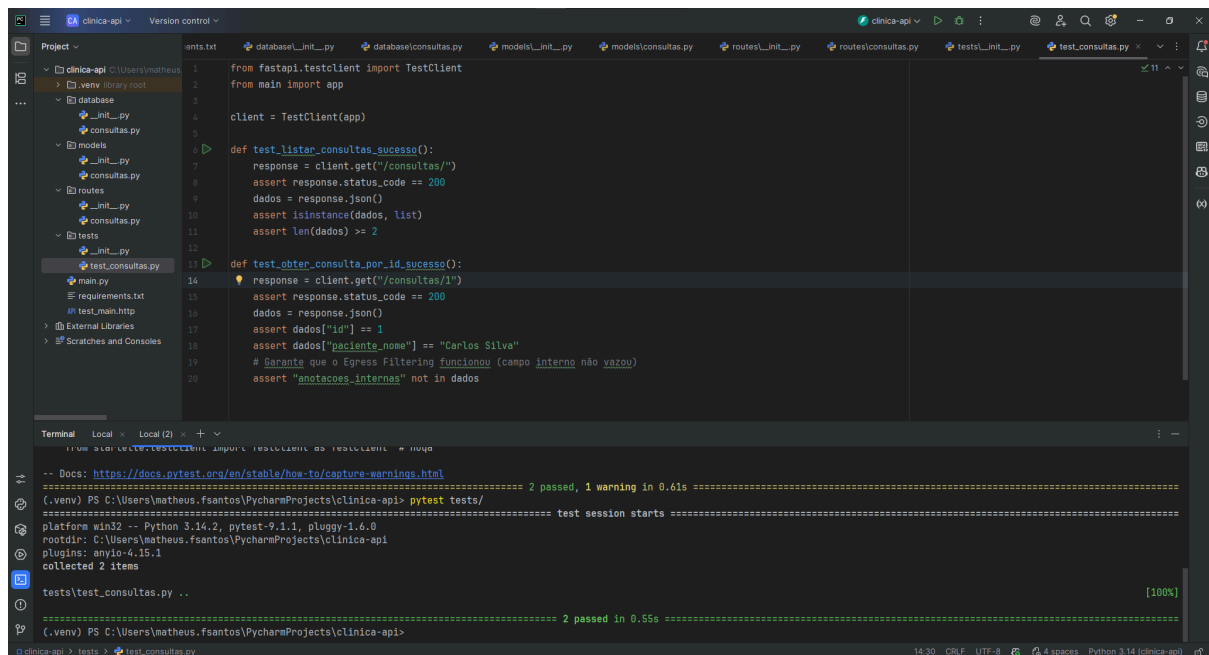
4. Evidências de Automação e Testes

Para garantir a integridade das defesas implementadas, o ambiente foi validado sob duas óticas:

Testes Automatizados (Pytest): Uma suíte de testes foi executada validando os caminhos de sucesso e o isolamento de dados. Resoluções de débitos técnicos (*DeprecationWarnings* do Starlette/httpx) foram aplicadas proativamente para garantir a estabilidade do CI/CD.

Testes HTTP Manuais (test_main.http): Bateria de testes operacionais comprovando o funcionamento prático:

- Resposta RESTful adequada (200 OK e 201 Created).
- Bloqueio bem-sucedido de payload inválido (422 Unprocessable Content).
- Resposta defensiva correta para IDs inexistentes (404 Not Found).



The screenshot displays the PyCharm IDE interface. The left sidebar shows the project structure with folders for 'database', 'models', 'routes', and 'tests'. The 'tests' folder is expanded, showing 'test_consultas.py' as the active file. The main editor window shows the following Python code:

```
1 from fastapi.testclient import TestClient
2 from main import app
3
4 client = TestClient(app)
5
6 def test_listar_consultas_sucesso():
7     response = client.get("/consultas/")
8     assert response.status_code == 200
9     dados = response.json()
10    assert isinstance(dados, list)
11    assert len(dados) >= 2
12
13 def test_obter_consulta_por_id_sucesso():
14     response = client.get("/consultas/1")
15     assert response.status_code == 200
16     dados = response.json()
17     assert dados["id"] == 1
18     assert dados["paciente_nome"] == "Carlos Silva"
19     # Garante que o Egress Filtering funciona (campo interno não vazou)
20     assert "anotacoes_internas" not in dados
```

The bottom terminal window shows the output of running 'pytest tests/'. The output indicates that 2 tests passed in 0.61s, with 1 warning. The warning is a DeprecationWarning from Starlette about the 'url_path_for' method. The test session starts on a Windows 32-bit platform using Python 3.14.2, pytest-9.1.1, and pluggy-1.6.0. The root directory is C:\Users\matheus.fsantos\PycharmProjects\clinica-api, and the plugins are anyio-4.15.1. The test file 'tests/test_consultas.py' is executed, and the results show 2 passed tests in 0.55s with a 100% success rate.

Project: clinica-api C:\Users\matheus

Files: .venv library root, database, _init_.py, consultas.py, models, _init_.py, consultas.py

API test_main.http

```
1 # Test your FastAPI endpoints
2
3 ### GET status da API (Raiz)
4 GET http://127.0.0.1:8000/
5 Accept: application/json
6
```

Services

HTTP Request

- GET test_main | GET consulta por ID (Sucesso) Status: 200 (12 ms)
- GET test_main | GET consulta por ID inexistente (404 Not Found) Status: 404
- GET test_main | GET status da API (Raiz) Status: 200 (5 ms)
- POST test_main | POST com payload inválido (422 Unprocessable Entity - falha)
- POST test_main | POST criando nova consulta (Sucesso - 201 Created) Status: 201

GET http://127.0.0.1:8000/

Show Request

HTTP/1.1 200 OK

(Headers) ...content-type: application/json...

```
{
  "status": "API da Clínica Médica operando normalmente."
}
```

Response file saved.

> 2026-09-29T213223.200.json

Response code: 200 (OK); Time: 5ms (5 ms); Content length: 56 bytes (56 B)

Project: clinica-api C:\Users\matheus

Files: .venv library root, database, _init_.py, consultas.py, models, _init_.py, consultas.py

API test_main.http

```
7 ### GET listando todas as consultas
8 GET http://127.0.0.1:8000/consultas/
9 Accept: application/json
10
```

Services

HTTP Request

- GET test_main | GET consulta por ID inexistente (404 Not Found) Status: 404
- GET test_main | GET listando todas as consultas Status: 200 (19 ms)
- GET test_main | GET status da API (Raiz) Status: 200 (5 ms)
- POST test_main | POST com payload inválido (422 Unprocessable Entity - falha)
- POST test_main | POST criando nova consulta (Sucesso - 201 Created) Status: 201

GET http://127.0.0.1:8000/consultas/

Show Request

```
{
  "id": 1,
  "paciente_nome": "Carlos Silva",
  "medico_nome": "Dr. Roberto",
  "data_hora": "2026-10-15T10:00:00",
  "especialidade": "Cardiologia",
  "status": "agendada"
},
{
  "id": 2,
  "paciente_nome": "Marina Souza",
  "medico_nome": "Dra. Ana",
  "data_hora": "2026-10-16T14:30:00",
  "especialidade": "Dermatologia",
  "status": "concluida"
},
{
  "id": 3,
  "paciente_nome": "Felipe Alves",
  "medico_nome": "Dr. Fernando",
  "data_hora": "2026-11-20T11:00:00",
  "especialidade": "Ortopedia",
  "status": "agendada"
}
]
```

Project: clinica-api C:\Users\matheus> .venv library root

- database
 - _init_.py
 - consultas.py
- models
 - _init_.py

test_main.http x main.py requirements.txt database_init_.py database\consultas.py models_init_.py

```
10
11 ### GET consulta por ID (Sucesso)
12 GET http://127.0.0.1:8000/consultas/1
13 Accept: application/json
14
```

Services

API HTTP Request

- GET test_main | GET consulta por ID (Sucesso) Status: 200 (20 ms)
- GET test_main | GET listando todas as consultas Status: 200 (19 ms)
- GET test_main | GET status da API (Raiz) Status: 200 (5 ms)
- POST test_main | POST com payload inválido (422 Unprocessable Entity - fal
- POST test_main | POST criando nova consulta (Sucesso - 201 Created) Statu

GET http://127.0.0.1:8000/consultas/1

Show Request

HTTP/1.1 200 OK

(Headers) ...content-type: application/json...

```
{
  "id": 1,
  "paciente_nome": "Carlos Silva",
  "medico_nome": "Dr. Roberto",
  "data_hora": "2026-10-15T10:00:00",
  "especialidade": "Cardiologia",
  "status": "agendada"
}
```

Response file saved.

> 2026-09-29T215624.200.json

Response code: 200 (OK); Time: 20ms (20 ms); Content length: 151 bytes (151 B)

Services

API HTTP Request

- GET test_main | GET consulta por ID (Sucesso) Status: 200 (20 ms)
- GET test_main | GET consulta por ID inexistente (404 Not Found) Status: 404
- GET test_main | GET listando todas as consultas Status: 200 (19 ms)
- GET test_main | GET status da API (Raiz) Status: 200 (5 ms)
- POST test_main | POST com payload inválido (422 Unprocessable Entity - fal

Accept: application/json

```
13
14
15 ### GET consulta por ID inexistente (404 Not Found)
16 GET http://127.0.0.1:8000/consultas/999
17 Accept: application/json
18
```

GET http://127.0.0.1:8000/consultas/999

Show Request

HTTP/1.1 404 Not Found

(Headers) ...content-type: application/json...

```
{
  "detail": "Consulta não encontrada."
}
```

Response file saved.

> 2026-09-29T215705.404.json

Response code: 404 (Not Found); Time: 2ms (2 ms); Content length: 37 bytes (37 B)

The screenshot shows the VS Code interface with a project named 'clinica-api'. The file explorer on the left shows the project structure. The main editor displays a Python file 'test_main.py' with a POST request to 'http://127.0.0.1:8000/consultas/'. The request body is a JSON object with patient name, doctor name, date, specialty, and status. The Services panel at the bottom shows the request details and the response, which is a 201 Created status with a JSON body containing the created consultation details.

```
19 ### POST criando nova consulta (Sucesso - 201 Created)
20 POST http://127.0.0.1:8000/consultas/
21 Content-Type: application/json
22
23 {
24     "paciente_nome": "Felipe Alves",
25     "medico_nome": "Dr. Fernando",
26     "data_hora": "2026-11-20T11:00:00",
27     "especialidade": "Ortopedia",
28     "status": "agendada"
29 }
```

Services

HTTP Request

test_main | GET consulta por ID (Sucesso) Status: 200 (20 ms)

test_main | GET consulta por ID inexistente (404 Not Found) Status: 404 (19 ms)

test_main | GET listando todas as consultas Status: 200 (19 ms)

test_main | POST com payload inválido (422 Unprocessable Entity - falta especialidade e status)

test_main | POST criando nova consulta (Sucesso - 201 Created) Status: 201 (6 ms)

POST http://127.0.0.1:8000/consultas/

HTTP/1.1 201 Created

(Headers) ..content-type: application/json..

```
{
  "id": 4,
  "paciente_nome": "Felipe Alves",
  "medico_nome": "Dr. Fernando",
  "data_hora": "2026-11-20T11:00:00",
  "especialidade": "Ortopedia",
  "status": "agendada"
}
```

Response file saved.

> 2026-09-29T215730.201.json

Response code: 201 (Created); Time: 6ms (6 ms); Content length: 150 bytes (150 B)

The screenshot shows the VS Code interface with a project named 'clinica-api'. The file explorer on the left shows the project structure. The main editor displays a Python file 'test_main.py' with a POST request to 'http://127.0.0.1:8000/consultas/'. The request body is a JSON object with patient name, doctor name, and date, but it is missing the 'especialidade' and 'status' fields. The Services panel at the bottom shows the request details and the response, which is a 422 Unprocessable Entity status with a JSON body indicating the missing fields.

```
31 ### POST com payload inválido (422 Unprocessable Entity - falta especialidade e status)
32 POST http://127.0.0.1:8000/consultas/
33 Content-Type: application/json
34
35 {
36     "paciente_nome": "F",
37     "medico_nome": "Dr. Fernando",
38     "data_hora": "2026-11-20T11:00:00"
39 }
```

Services

HTTP Request

test_main | GET consulta por ID (Sucesso) Status: 200 (20 ms)

test_main | GET consulta por ID inexistente (404 Not Found) Status: 404 (19 ms)

test_main | GET listando todas as consultas Status: 200 (19 ms)

test_main | POST com payload inválido (422 Unprocessable Entity - falta especialidade e status)

test_main | POST criando nova consulta (Sucesso - 201 Created) Status: 201 (6 ms)

POST http://127.0.0.1:8000/consultas/

HTTP/1.1 422 Unprocessable Entity

(Headers) ..content-type: application/json..

```
{
  "type": "missing",
  "loc": [
    "body",
    "status"
  ],
  "msg": "Field required",
  "input": {
    "paciente_nome": "F",
    "medico_nome": "Dr. Fernando",
    "data_hora": "2026-11-20T11:00:00"
  }
}
```

Response file saved.

> 2026-09-29T215747.422.json

Resumo da Implementação - Exercício 2

Enunciado:

Implemente response models Pydantic que controlem exatamente os campos expostos na API de consultas, e implemente a página HTML especificada com Jinja2 (usando herança de templates), exibindo dados de consultas de forma segura contra XSS.

1. Objetivo

O Exercício 2 teve como foco principal o controle rigoroso da exposição de dados (Egress Filtering) em endpoints JSON e a implementação de uma arquitetura de templates HTML seguros no lado do servidor. O objetivo central foi blindar a aplicação contra as vulnerabilidades de Vazamento de Informações Sensíveis (Information Disclosure) e *Cross-Site Scripting* (XSS) Refletido/Armazenado.

2. Arquitetura de Templates e Herança (Jinja2)

Para a interface visual da recepção da clínica, a renderização foi estruturada utilizando o motor Jinja2 nativo do FastAPI. A organização seguiu o princípio de DRY (Don't Repeat Yourself) através da herança de templates.

- **templates/base.html:** Atua como o template pai, concentrando o cabeçalho, rodapé e as declarações estruturais de segurança. A utilização desta herança centraliza o controle visual e evita a duplicação de código.
- **templates/agenda.html:** Estende a base e implementa apenas o bloco dinâmico (`{% block content %}`) necessário para iterar sobre a lista de consultas e exibi-las na tabela.
- **Isolamento de Rotas:** O fluxo da interface gráfica (GET `/consultas/agenda-html`) foi isolado do fluxo de dados da API REST, garantindo que o acoplamento não afete os clientes que consomem apenas JSON.

3. Controles de Segurança Aplicados

3.1. Egress Filtering e Prevenção de Vazamento de Dados

Foi configurado o modelo de saída `ConsultaResponse` e rigidamente vinculado ao parâmetro `response_model` dos endpoints JSON.

- **Controle:** O FastAPI intercepta o retorno da função e o filtra estruturalmente antes de serializar o JSON final. Apenas os campos definidos no esquema (ID, Paciente, Médico, Data, Especialidade, Status) são autorizados a passar.
- **Mitigação:** Evita a fraqueza conhecida como CWE-200 (Exposição de Informação Sensível para Ator Não Autorizado). O campo interno restrito `anotacoes_internas`, que poderia conter dados médicos cobertos pela

LGPD, é sistematicamente removido antes de atingir a fronteira externa, frustrando a tática de *Discovery* de possíveis atacantes.

3.2. Prevenção contra XSS (Cross-Site Scripting)

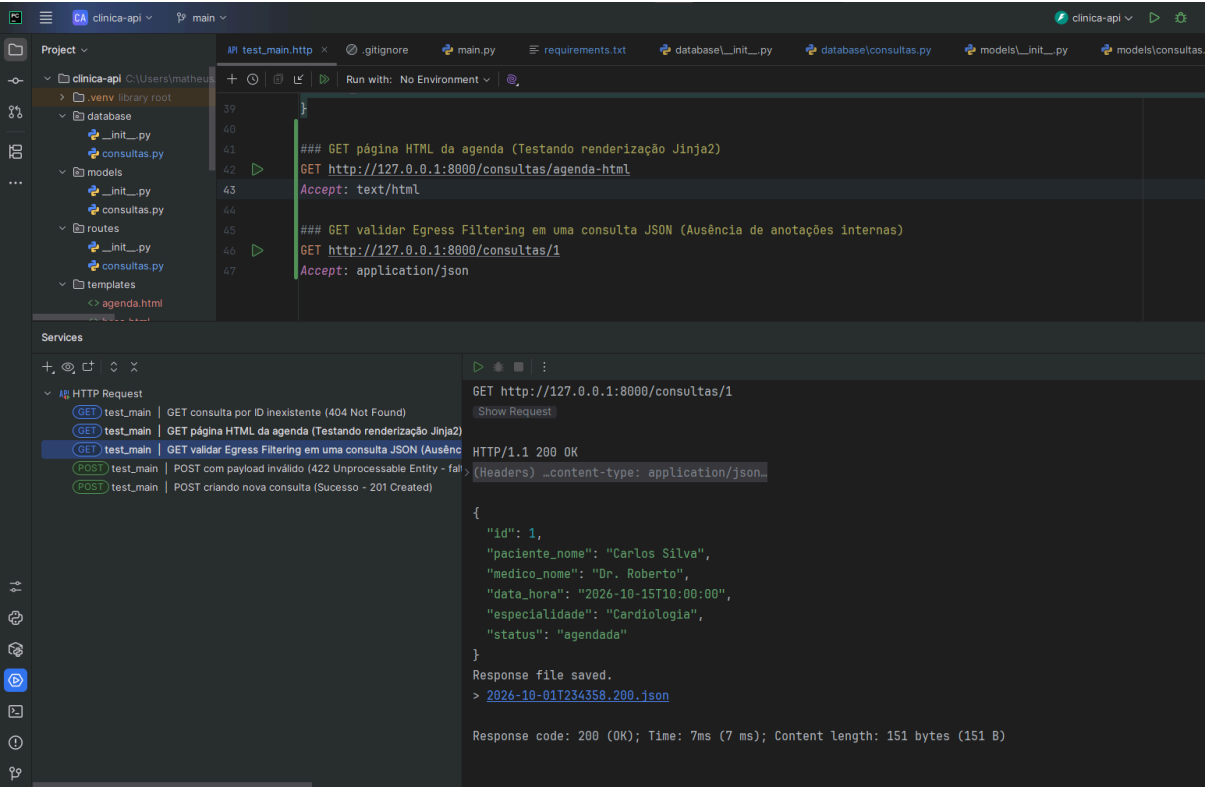
O front-end gerado com Jinja2 mitiga proativamente a injeção de scripts maliciosos.

- **Controle:** Renderizar inputs sem tratamento permite ataques de XSS, onde scripts maliciosos injetados são executados no navegador das vítimas. O motor Jinja2 (padrão no FastAPI) trata entradas maliciosas convertendo tags HTML automaticamente, processo conhecido como *auto-escape*.
- **Mitigação:** Esse controle converte caracteres sensíveis (como < e >) em entidades HTML (< e >). Assim, o navegador interpreta o payload malicioso apenas como texto visual inofensivo, impedindo a execução de qualquer código JavaScript indevido.

4. Evidências de Automação e Testes

Para validar a integridade e a resiliência das novas defesas, a suíte de testes foi ampliada com cenários focados em segurança:

1. **Testes Automatizados (Pytest):**
 - a. Validação de Egress Filtering: O teste `test_garantir_ao_vazamento_anotacoes_internas_json` consumiu a rota REST e verificou via `assert` a ausência completa da chave sensível no payload JSON.
 - b. Validação Anti-XSS: O teste `test_renderizacao_agenda_html_sucesso_e_seguranca` forçou a injeção de um payload malicioso (`<script>alert(...)</script>`) em um recurso interno simulado e realizou o parse da resposta HTML, comprovando a neutralização do vetor de ataque pelo *auto-escape* do Jinja2.
2. **Testes HTTP Operacionais (test_main.http):** Foram executadas requisições manuais provando o funcionamento prático da renderização HTML para o cliente final e a sanitização dos endpoints de consumo de dados.



Relatório de Arquitetura de Segurança - Exercício 3

Enunciado:

Avalie a aplicação construída sob a ótica da tríade CIA, mapeie frameworks de referência a controles concretos já implementados e construa um DFD básico identificando trust boundaries e fluxos de dados sensíveis (em especial, dados de pacientes).

1. Avaliação da Tríade CIA (Confidencialidade, Integridade e Disponibilidade)

A avaliação estrutural da clinica-api demonstra a aplicação proativa de controles de segurança desde as primeiras fases do ciclo de desenvolvimento, embora existam riscos arquitetônicos residuais comuns em MVPs (Minimum Viable Products).

A **Confidencialidade** possui um peso regulatório crítico e inegociável neste projeto. Por se tratar de um sistema de saúde, os dados manipulados (como especialidades médicas e anotações clínicas) são classificados como **Dados Pessoais Sensíveis** pela LGPD (Lei Geral de Proteção de Dados - Art. 5º, inciso II). O vazamento de informações contidas no campo `anotacoes_internas` não configuraria apenas uma falha técnica, mas uma grave infração regulatória sujeita a sanções.

Pilar	Situação Atual (O que já está implementado)	Lacuna Real Observável no Código
Confidencialidade	Implementação de Egress Filtering rigoroso com <code>response_model</code> (<code>ConsultaResponse</code>) no Pydantic, garantindo que o campo sensível <code>anotacoes_internas</code> seja omitido nas respostas JSON.	Falta de Autenticação/Autorização : A API é totalmente pública. Qualquer usuário anônimo pode listar todas as consultas e ver nomes de pacientes e médicos nos endpoints <code>/consultas/</code> e <code>/consultas/agenda-html</code> .
Integridade	Validação estrita de Ingress Filtering (tipos e tamanhos via <code>ConsultaCreate</code> com <code>Field</code> do Pydantic) rejeitando <i>payloads</i> malformados. Uso do Jinja2 com auto-escape nativo mitigando injeção de scripts (XSS) no frontend.	Falta de Persistência Segura / Mutabilidade : O armazenamento atual é um dicionário em memória (<code>consultas_db</code>). Não há logs de auditoria imutáveis ou transações em um banco de dados real que garantam a integridade atômica da informação.
Disponibilidade	Utilização de rotas assíncronas (<code>async def</code>) no FastAPI e tratamento padronizado de exceções (ex: <code>HTTPException 404</code>), evitando que requisições inválidas causem um <i>crash</i> no servidor.	Ausência de Paginação e Rate Limiting : Um atacante pode realizar uma negação de serviço (DoS) enviando requisições massivas de POST ou solicitar todos os dados do banco simultaneamente, sobrecarregando a memória da aplicação.

2. Mapeamento de Frameworks de Referência

A tabela abaixo correlaciona as práticas implementadas na clinica-api com os padrões da indústria exigidos em esteiras DevSecOps, demonstrando a conformidade técnica da aplicação.

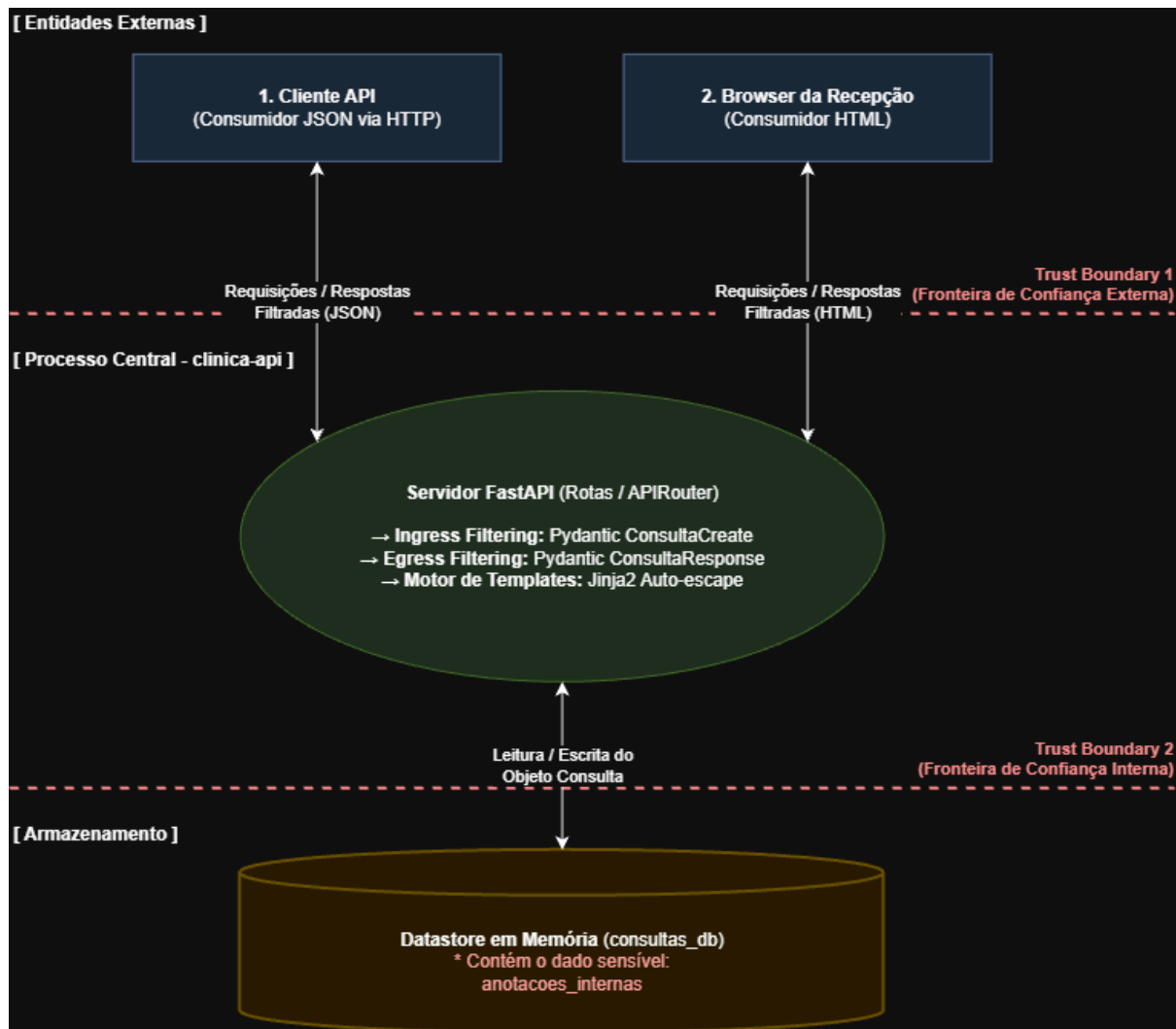
Framework de Referência	Controle de Segurança Concreto Aplicado	Justificativa / Aplicação na Prática
OWASP (Foco em Vulnerabilidades Web)	1. Auto-escape do Jinja2: 2. response_model do Pydantic:	1. O Jinja2 trata entradas maliciosas convertendo caracteres sensíveis em entidades HTML. Isso mitiga diretamente falhas de Cross-Site Scripting (XSS) (OWASP Top 10 - A03:2021-Injection). 2. A filtragem restringe o retorno de dados desnecessários, prevenindo o risco de Exposição Excessiva de Dados (OWASP API3:2023).
NIST SSDF (Ciclo de Vida de Desenvolvimento Seguro)	Uso de Ambiente Virtual (venv) e requirements.txt	O NIST SSDF exige a proteção do ambiente de desenvolvimento e da cadeia de suprimentos (prática PS: Protect Software). Isolar dependências garante que a aplicação não sofra com conflitos ou dependa de bibliotecas globais não versionadas e potencialmente inseguras.

MITRE (Táticas, Técnicas e Fraquezas - ATT&CK e CWE)	Filtro de Saída (Egress Filtering) com Pydantic	O uso estrito do modelo de saída evita a fraqueza conhecida como CWE-200 (Exposição de Informação Sensível para Ator Não Autorizado) . No framework MITRE ATT&CK, impedir esse tipo de vazamento de informações de <i>backend</i> frustra ativamente a tática de Discovery , dificultando o reconhecimento do ambiente por parte de um invasor.
--	--	---

3. Construção e Análise do Data Flow Diagram (DFD)

A modelagem de ameaças (Threat Modeling) exige a identificação clara de como os dados transitam entre os domínios de confiança. O diagrama abaixo representa o DFD focado no tratamento dos dados da clinica-api.

Representação do Diagrama (Modelo de Blocos)



Análise do Fluxo de Dados Sensível

O fluxo de dados crítico avaliado consiste no trajeto do objeto Consulta, especificamente seu atributo confidencial `anotacoes_internas`.

- Recuperação e Tráfego Interno:** Quando uma requisição GET é feita pelo cliente, a API (Processo) consulta o `consultas_db` (Armazenamento). O objeto completo (incluindo o dado sensível da LGPD) cruza a **Trust Boundary 2 (Fronteira Interna)** para a memória de processamento da API. Este tráfego interno é considerado de baixo risco, pois ocorre dentro do domínio seguro do servidor.
- Interceptação (Security Gate):** O ponto crítico de segurança ocorre na **Trust Boundary 1 (Fronteira Externa)**, que separa a API do Cliente. É exatamente antes de cruzar esta fronteira que o framework atua.
- Atuação do Filtro:** O `response_model=ConsultaResponse` atua como um inspetor de alfândega (Security Gate) em tempo de execução. Ele serializa a resposta interceptando o dicionário bruto da Consulta,

permitindo a passagem apenas dos atributos explicitamente declarados no modelo de saída. O campo `anotacoes_internas` é cirurgicamente podado, impedindo sumariamente o vazamento da informação para a rede pública. Na rota web (`agenda-html`), o motor Jinja2 age de forma análoga garantindo que, mesmo que o dado fosse repassado, a renderização mitigaria injeções indesejadas na interface.

Relatório de Segurança Arquitetural - Exercício 4

Enunciado:

Elabore misuse cases relevantes para a aplicação, aplique STRIDE a pelo menos três componentes e construa o threat model consolidado, mapeando ativos, superfícies de ataque e mitigações.

1. Elaboração e Priorização de Misuse Cases (Casos de Abuso)

Para compreender as vulnerabilidades da clinica-api sob a perspectiva do atacante, elaboramos casos de abuso (Misuse Cases) focados nas regras de negócio e nas fronteiras de confiança (Trust Boundaries) da aplicação.

Catálogo de Misuse Cases

- **MC-01: Leitura e Vazamento de Anotações Médicas Privadas (BOLA / IDOR)**
 - **Ator Malicioso:** Paciente Mal-intencionado / Atacante Autenticado.
 - **Vetor de Ataque:** Manipulação direta do parâmetro `id` na rota `GET /consultas/{id}` (ex: iterar IDs sequenciais) para tentar visualizar agendamentos de terceiros.
 - **Impacto no Negócio:** Information Disclosure e violação gravíssima da LGPD (vazamento de PHI - *Protected Health Information*). Perda irreparável de confidencialidade e multas regulatórias severas.
- **MC-02: Sequestro de Sessão via Injeção de Scripts Maliciosos (Stored XSS)**
 - **Ator Malicioso:** Atacante Anônimo.
 - **Vetor de Ataque:** Submissão de *payloads* JavaScript maliciosos (ex: `<script>alert('Roubo de Cookies')</script>`) nos campos `paciente_nome` ou `especialidade` através do endpoint `POST`

/consultas/. O script é persistido e executado quando a recepção acessa o painel web.

- **Impacto no Negócio:** Elevação de Privilégios (*Elevation of Privilege*). O atacante pode sequestrar a sessão do administrador da recepção, burlando controles de acesso.
- **MC-03: Alteração de Estado via Atribuição em Massa (BOPLA / Mass Assignment)**
 - **Ator Malicioso:** Paciente Mal-intencionado / Insider.
 - **Vetor de Ataque:** Injeção de chaves JSON não documentadas durante a criação ou atualização da consulta (ex: "status": "concluida", "anotacoes_internas": "Paciente VIP") explorando o forte acoplamento entre a entrada e o modelo de persistência.
 - **Impacto no Negócio:** Quebra de Integridade (*Tampering*). Modificação indevida de fluxos operacionais e registros médicos sem autorização prévia.
- **MC-04: Negação de Serviço e Exaustão de Recursos (DoS / Brute Force)**
 - **Ator Malicioso:** Atacante Anônimo / *Botnets* / *Scalper Bots*.
 - **Vetor de Ataque:** Envio massivo e concorrente de milhares de requisições automatizadas para os endpoints GET /consultas/ ou POST /consultas/, explorando a ausência de limitação de taxa (*Rate Limiting*).
 - **Impacto no Negócio:** Quebra de Disponibilidade (*Denial of Service*). O sistema de agendamento fica indisponível para pacientes legítimos e trava o painel operacional da recepção.

Matriz de Priorização (Impacto e Risco)

Prioridade	ID do Misuse Case	Risco	Justificativa Arquitetural

1 (Crítica)	MC-01 (BOLA)	Alto	Impacto direto e catastrófico sob a ótica da LGPD (vazamento de PHI). A probabilidade de enumeração de IDs numéricos previsíveis em APIs REST é altíssima sem defesas adequadas.
2 (Alta)	MC-02 (XSS)	Alto	Compromete a camada administrativa (recepção) inteira. Embora demande interação da vítima (abrir a página), o impacto de uma elevação de privilégio lateral justifica a alta criticidade.
3 (Média)	MC-03 (BOPLA)	Médio	Fere a integridade dos dados, porém a probabilidade de um estrago maior é mitigada parcialmente pelos esquemas iniciais de <i>Ingress Filtering</i> do Pydantic já implementados no Exercício 1.

4 (Baixa)	MC-04 (DoS)	Médio	Alta probabilidade de ocorrência por <i>bots</i> , mas o impacto é temporário (apenas indisponibilidade), não resultando em vazamento de dados de saúde.
------------------	--------------------	--------------	--

2. Aplicação do Framework STRIDE aos Componentes da API

Para garantir que todas as classes de ameaças sejam mapeadas, aplicamos o modelo STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) aos limites de confiança estabelecidos no DFD (Exercício 3).

Componente	Categoria STRIDE	Ameaça Concreta Identificada	Impacto Potencial
1. Endpoints RESTful (routes/consultas.py)	Spoofing	Falsificação de identidade em requisições de API na ausência de validação de <i>tokens</i> JWT assinados.	Acesso não autenticado a endpoints restritos de agendamento.
1. Endpoints RESTful (routes/consultas.py)	Denial of Service	<i>Flood</i> de requisições de criação de agendamento sem <i>Rate Limiting</i> ou paginação na listagem.	Exaustão da <i>thread pool</i> do FastAPI; API da clínica fica <i>offline</i> .
2. Painel HTML Jinja2 (templates/agenda.html)	Elevation of Privilege	Execução de <i>Stored XSS</i> no navegador do funcionário da recepção,	Sequestro da sessão da recepção (Admin); controle

Componente	Categoria STRIDE	Ameaça Concreta Identificada	Impacto Potencial
		contornando defesas via <i>Frontend</i> .	total do painel de operações.
3. Camada de Banco/Mock (database/consultas.py)	Tampering	Injeção de atributos restritos (ex: <i>status</i>) explorando a ausência de <i>extra='forbid'</i> (BOPLA/Mass Assignment).	Alteração indevida de dados clínicos ou burla de processos de faturamento.
3. Camada de Banco/Mock (database/consultas.py)	Information Disclosure	Vazamento estrutural de <i>anotacoes_internas</i> caso o <i>Egress Filtering</i> do <i>ConsultaResponse</i> venha a falhar no futuro.	Quebra de sigilo médico; violação da LGPD e penalizações legais.
3. Camada de Banco/Mock (database/consultas.py)	Repudiation	Inexistência de trilha de auditoria (Logs centralizados) detalhando quem, quando e de onde uma consulta foi alterada.	Impossibilidade técnica de responsabilizar agentes internos em caso de fraude.

3. Threat Model Consolidado (Ativos, Superfícies e Mitigações)

O modelo consolidado abaixo estabelece a ponte direta entre as ameaças mapeadas e os controles técnicos arquiteturais que comporão as próximas etapas de desenvolvimento (Implementação e Infraestrutura), seguindo o *OWASP API Security Top 10*.

Ativo Protegido	Superfície de Ataque	Categoria STRIDE	Ameaça Específica	Controle / Mitigação Proposta (Arquitetura/Código)
Dados Sensíveis de Pacientes (PHI/PII)	Endpoints de Leitura (GET /{id}) e Listagem	Information Disclosure	<i>Broken Object Level Authorization</i> (BOLA) e vazamento de notas internas.	Manutenção estrita do <i>Egress Filtering</i> via <code>response_model</code> (Pydantic). Implementação de checagem cruzada garantindo que o <code>user_id</code> do Token JWT corresponda ao dono do recurso.
Integridade da Agenda Médica	Endpoint de Criação/Edição (POST /, PUT)	Tampering	<i>Mass Assignment</i> (BOPLA) injetando propriedades não documentadas.	Reforço do <i>Ingress Filtering</i> utilizando a configuração <code>extra='forbid'</code> no Pydantic v2. Adoção de modelo parametrizado com <code>SQLModel</code> para evitar <i>SQL Injection</i> .
Sessão/Painel da Recepção	Rota Web de Renderização (GET /agenda-html)	Elevation of Privilege	Execução de artefatos maliciosos via <i>Stored XSS</i> ou <i>CSRF</i> .	Manter a sanitização e o Auto-escape estrito ativado nativamente no motor Jinja2 . Implementação de <i>Security Headers</i> (CSP, X-Content-Type-Options) na resposta HTTP.
Disponibilidade do Servidor	Todos os Endpoints da API	Denial of Service	Exaustão por varredura bruta (<i>Fuzzing/Brute Force</i>) automatizada.	Implementação de limitação de volumetria (<i>Rate Limiting</i>) no <i>Gateway</i> e via biblioteca <code>slowapi</code> no FastAPI.

Ativo Protegido	Superfície de Ataque	Categoria STRIDE	Ameaça Específica	Controle / Mitigação Proposta (Arquitetura/Código)
				Bloqueio de tamanho de <i>payload</i> via Pydantic.
Credenciais / Tokens	Fluxo de Login e Interceptação de Rede	Spoofing / Repudiation	Fixação de Sessão, Roubo de Tokens e Falta de Auditoria.	Adoção de OAuth2 com <i>Tokens</i> JWT para controle de acesso baseado em papéis (RBAC). Emprego de <i>PKCE</i> para mitigar interceptação e geração de <i>Logs</i> estruturados via opentelemetry

Relatório de Segurança Arquitetural - Exercício 5

Enunciado:

Particione o sistema em componentes, mapeie o fluxo de dados entre eles e identifique vetores de ataque relevantes nos três eixos de segurança de APIs.

1. Mapeamento de Partições e Fronteiras de Segurança

Para aplicar o princípio do Defense in Depth (Defesa em Profundidade) e proteger os dados sensíveis dos pacientes (PHI/LGPD), a arquitetura da clinica-api foi dividida em três partições lógicas isoladas por Fronteiras de Segurança (Trust Boundaries).

Arquitetura de Partições

- **Partição 1: Borda / Clientes Externos (Untrusted Zone):** Representa o ambiente fora do controle direto da API. Inclui o Frontend (Web/Mobile) consumindo JSON, o *Browser* utilizado no Painel da Recepção (HTML) e os sistemas automatizados (M2M) do Laboratório Parceiro. É a superfície inicial de ataque.

- **Partição 2: Aplicação / Core Business (Trusted Zone):** O motor de processamento central da clínica. Engloba o Servidor FastAPI, as rotas (APIRouter), os motores de validação de entrada/saída (Pydantic v2) e o motor de renderização de templates nativo (Jinja2).
- **Partição 3: Persistência / Dados (Restricted Zone):** A camada de armazenamento de dados, atualmente mapeada no datastore em memória (consultas_db.py), desenhada para acoplar motores de persistência reais via ORM (como SQLite/SQLModel). Contém informações críticas de auditoria e dados médicos (anotacoes_internas).

Fronteiras de Segurança (Security Boundaries) e Validação de Fluxo

1. Trust Boundary 1 (Borda Externa ↔ Aplicação):

Esta é a fronteira mais crítica. Quando uma requisição cruza essa linha, a aplicação assume que o dado é malicioso.

- Ingress Filtering (Entrada):** A requisição é imediatamente interceptada pelo modelo ConsultaCreate (Pydantic v2), que impõe tipagem forte, limitação de caracteres (min_length, max_length) e rejeita propriedades não esperadas, impedindo *Mass Assignment*.
- Egress Filtering (Saída):** Antes da resposta JSON retornar à Partição 1, o modelo ConsultaResponse serializa o objeto, podendo ativamente campos sensíveis (como anotacoes_internas) e mitigando a *Excessive Data Exposure*.
- Sanitização Visual:** Para a interface da recepção, a resposta cruza a fronteira através do TemplateResponse (Jinja2), que aplica *auto-escape* nativo, neutralizando *cargas* de *Cross-Site Scripting* (XSS) injetadas no banco.

2. Trust Boundary 2 (Aplicação ↔ Persistência):

Separa a lógica de negócios do banco de dados. A validação aqui garante que a comunicação com o banco seja parametrizada (através do uso de ORMs como o SQLModel), bloqueando ataques de *SQL Injection* (SQLi) e garantindo que o acesso a registros seja validado contra o escopo do usuário autenticado no contexto da transação.

2. Mapeamento de Vetores de Ataque nos Três Eixos de Segurança de APIs

Garantir a segurança apenas no código é insuficiente. A tabela abaixo analisa o cenário de exposição da API para múltiplos consumidores com base nos três eixos estruturais de Segurança de APIs.

Eixo de Segurança	Vetor de Ameaça / Risco Mapeado	Controle Arquitetural / Solução de Defesa na clinica-api
a) Eixo de Design (Decisões de Projeto e Arquitetura)	BOLA / IDOR: Acesso não autorizado a consultas de outros pacientes via manipulação de ID numérico. Excessive Data Exposure: Vazamento de PII/PHI (ex: anotacoes_internas).	Defesa: Aplicação rigorosa do <i>Egress Filtering</i> utilizando <code>response_model=ConsultaResponse</code> . Adoção de controle de acesso (RBAC/ABAC) via <i>claims</i> no JWT e isolamento de acessos M2M (Laboratório) utilizando <i>scopes</i> rígidos e segregados.
b) Eixo de Implementação (Práticas Seguras de Código)	Mass Assignment (BOPLA): Injeção de status indevidos (ex: forçar "concluida"). Stored/Reflected XSS: Injeção de scripts no painel da recepção. SQL Injection (SQLi): Abuso de <i>queries</i> mal formadas.	Defesa: <i>Ingress Filtering</i> com <code>extra='forbid'</code> no modelo Pydantic v2. Renderização de frontend web herdando <code>base.html</code> com Auto-escape estrito do Jinja2 . Utilização de consultas estritamente parametrizadas via abstração de ORM (SQLModel).
c) Eixo de Infraestrutura e Borda (Proteção do Ambiente de Execução)	Fuzzing / DoS: Exaustão de recursos do servidor. Spoofing / MitM: Interceptação de tokens em trânsito e abuso de cabeçalhos frouxos.	Defesa: Aplicação de <i>Rate Limiting</i> (<code>slowapi</code>), CORS configurado com <i>Allowlist</i> explícita (sem uso de curingas *), <i>Security Headers</i> (HSTS, X-Frame-Options, X-Content-Type-Options) e gestão segura de segredos via <code>BaseSettings / .env</code> (Pydantic Settings) fora do código-fonte.

3. Matriz de Exposição para Consumidores Externos

Ao abrir a clinica-api para integração externa (pacientes, recepção e laboratório parceiro), a superfície de ataque é expandida. A matriz abaixo define os limites seguros e os mecanismos de controle de acesso exigidos para cada tipo de consumidor.

Consumidor Externo	Interface Utilizada	Nível de Exposição	Modelo de Autenticação / Controle Proposto	Dados Acessíveis
1. Frontend Web/Mobile (App de Pacientes)	JSON REST API (Endpoints /consultas - GET, POST)	Alto (Internet Pública)	OAuth2 / JWT (Authorization Code Flow / PKCE): Usuários autônomos recebem tokens efêmeros (Bearer) via Injeção de Dependências (Depends) no FastAPI.	Restrito (Self-Data): Somente campos presentes no ConsultaResponse. LGPD respeitada: o paciente só acessa as consultas que o pertencem (Prevenção BOLA).
2. Painel da Recepção (Interface Administrativa Interna)	HTML Renderizado via Motor Jinja2 (Endpoint /consultas/agenda-html)	Médio (Intranet / Rede Corporativa)	Sessão Baseada em Cookie (HTTPOnly / Secure): Associado a validações de <i>Tokens</i> <i>Anti-CSRF</i> nas requisições estatais. Autenticação ligada ao provedor de	Amplio (Sanitizado): Acesso visual a todos os agendamentos do dia em formato de tabela, blindado contra injeções. Campos internos médicos (anotacoes_internas)

Consumidor Externo	Interface Utilizada	Nível de Exposição	Modelo de Autenticação / Controle Proposto	Dados Acessíveis
			identidade (Ex: Keycloak).	permanecem ocultos.
3. Laboratório Parceiro (Integração M2M B2B)	JSON REST API (Consumo via <i>scripts</i> e Webhooks)	Baixo a Médio (Comunicação Server-to-Server)	OAuth2 Client Credentials Flow: Autenticação automatizada focada na aplicação sem interação humana. Uso estrito de permissões granulares (scope="appointments:read") limitadas por API Gateway.	Somente Leitura (Read-Only): Limitado à visualização do status de consultas e especialidade via ConsultaResponse. Interações de gravação ou deleção (POST, DELETE) são rejeitadas pelo RBAC.

Relatório de Implementação - Exercício 6

Enunciado:

Particione o sistema em componentes, mapeie o fluxo de dados entre eles e identifique vetores de ataque relevantes nos três eixos de segurança de APIs.

1. Objetivo

O Exercício 6 teve como foco erradicar vulnerabilidades críticas mapeadas no OWASP API Security Top 10, estabelecendo o controle de acesso robusto com JWT (JSON Web Tokens), armazenamento seguro de senhas com hashing unidirecional e a construção de Security Gates para prevenir ataques de escalonamento de privilégios e manipulação indevida de dados.

2. Arquitetura e Modularização de Segurança

Para isolar a lógica de segurança das regras de negócio, o sistema foi expandido com um novo módulo criptográfico, mantendo a coesão da arquitetura:

- **auth/:** Novo módulo central contendo:
 - **hash_password.py:** Gerenciamento de criptografia utilizando Passlib e o algoritmo Bcrypt.
 - **jwt_handler.py:** Geração e decodificação segura de tokens JWT utilizando python-jose, com expiração baseada em datas UTC modernas (mitigando DeprecationWarnings).
 - **authenticate.py:** Injeção de dependência (Depends) com OAuth2PasswordBearer, atuando como middleware validador de acesso.
- **routes/user.py & routes/admin.py:** Controladores dedicados à emissão de tokens, verificação MFA (Múltiplo Fator) e exposição de rotas restritas baseadas em papéis.

3. Controles de Segurança Aplicados

A implementação seguiu o princípio de *Defense in Depth* (Defesa em Profundidade), rejeitando a confiança implícita nos dados enviados pelo cliente e baseando a autorização exclusivamente em *claims* criptografados no servidor.

- **3.1. Hashing Criptográfico e Autenticação Segura**
 - **Controle:** Senhas nunca são armazenadas em texto plano. O algoritmo Bcrypt (com *work factor* adaptativo) foi implementado para gerar o hash das credenciais no mock do banco de dados (users_db).
 - **Mitigação:** Protege contra vazamento de credenciais e ataques de força bruta/Rainbow Tables em caso de comprometimento da base de dados.
- **3.2. Prevenção contra BFLA e Implementação de MFA**
 - **Controle:** Rotas administrativas (como GET /admin/dashboard) foram blindadas pela dependência `require_admin`. O sistema verifica rigorosamente o papel (`role == "admin"`) e exige a verificação em duas etapas (`mfa_verified == True`) embutidas no token JWT.
 - **Mitigação:** Erradica o BFLA (Broken Function Level Authorization), impedindo o escalonamento vertical. Um usuário com perfil de "médico", mesmo que descubra a rota de administrador, é

sumariamente bloqueado com um erro 403 Forbidden pelo backend, sem depender de ocultação visual no frontend.

- **3.3. Prevenção contra BOLA (Ownership de Recursos)**

- **Controle:** Na edição e exclusão de consultas (PUT e DELETE), a API extrai a identidade do usuário diretamente da *claim* sub do token JWT autenticado e a compara com o *medico_id* vinculado à consulta no banco de dados.
- **Mitigação:** Mitiga o BOLA (Broken Object Level Authorization), prevenindo o escalonamento horizontal. Se o Médico B tentar modificar uma consulta do Médico A alterando o ID na URL, o sistema rejeita a ação com um falso 404 Not Found, evitando vazamento de informações (Information Disclosure) sobre a existência do recurso.

- **3.4. Prevenção contra BOPLA (Mass Assignment)**

- **Controle:** O modelo de entrada Pydantic (ConsultaCreate) foi refatorado para **não** aceitar o campo *medico_id* vindo do payload JSON do cliente. A identidade do médico é injetada nos dados de forma implícita e confiável no backend pelo JWT.
- **Mitigação:** Impede ataques de *Mass Assignment*, impossibilitando que um atacante forge o dono da consulta ao enviar campos adicionais não documentados no corpo da requisição.

4. Evidências de Automação e Testes

Para comprovar a eficácia das defesas de AuthN/AuthZ, o ambiente foi submetido a baterias de testes focados em simulação de ataques:

- **Testes Automatizados (Pytest):** O teste *test_bloqueio_usuario_nao_admin_em_rota_restrita_admin* foi executado com sucesso (100% pass), simulando o login de um médico e comprovando o bloqueio estrutural (403 Forbidden) ao tentar invadir o escopo do administrador.
- **Testes HTTP Interativos (test_main.http):** Simulação ponta a ponta dos fluxos de ataque e defesa:
 - Sucesso na exigência de MFA para contas de alto privilégio (Admin).
 - Captura e gerenciamento dinâmico de Tokens Bearer via script global da IDE.
 - Validação BOLA: Bloqueio (404 Not Found) comprovado ao tentar alterar/excluir registros de terceiros utilizando um token de permissão paralela.

Project Explorer: auth (authenticate.py, hash_password.py, jwt_handler.py), database (_init_.py, consultas.py, users.py), models (_init_.py, consultas.py), routes (_init_.py, admin.py, consultas.py).

Run with: No Environment

```
94
95 ### 3. SUCESSO: Acessar Rota Admin com Token Admin (MFA Verificado)
96 GET http://127.0.0.1:8000/admin/dashboard
97 Authorization: Bearer {{token_admin}}
98
99 ### 4. DEFESA BFLA: Acessar Rota Admin com Token de Médico (Retorna 403 Forbidden)
100 GET http://127.0.0.1:8000/admin/dashboard
101 Authorization: Bearer {{token_medico_a}}
102
103
104 ### =====
105 ### EXERCÍCIO 6 - TESTES BOLA/IDOR (OWNERSHIP DE CONSULTAS)
```

Services

HTTP Request

Method	URL	Status	Time
POST	test_main	loginAdminMFA	Status: 200 (4 ms)
POST	test_main	loginMedicoA	Status: 200 (301 ms)
POST	test_main	loginMedicoB	Status: 200 (302 ms)
GET	test_main	3. SUCESSO: Acessar Rota Admin com Token Admin (MFA V	

GET http://127.0.0.1:8000/admin/dashboard

Show Request

HTTP/1.1 200 OK

(Headers) ...content-type: application/json...

```
{
  "message": "Bem-vindo ao Painel de Controle (Admin).",
  "admin_id": "00000000-0000-0000-0000-000000000000"
}
```

Response file saved.

> [2026-10-02T234507.200.json](#)

Response code: 200 (OK); Time: 6ms (6 ms); Content length: 104 bytes (104 B)

Project Explorer: auth (authenticate.py, hash_password.py, jwt_handler.py), database (_init_.py, consultas.py, users.py), models (_init_.py, consultas.py), routes (_init_.py, admin.py, consultas.py).

Run with: No Environment

```
94
95 ### 3. SUCESSO: Acessar Rota Admin com Token Admin (MFA Verificado)
96 GET http://127.0.0.1:8000/admin/dashboard
97 Authorization: Bearer {{token_admin}}
98
99 ### 4. DEFESA BFLA: Acessar Rota Admin com Token de Médico (Retorna 403 Forbidden)
100 GET http://127.0.0.1:8000/admin/dashboard
101 Authorization: Bearer {{token_medico_a}}
102
103
104 ### =====
105 ### EXERCÍCIO 6 - TESTES BOLA/IDOR (OWNERSHIP DE CONSULTAS)
```

Services

HTTP Request

Method	URL	Status	Time
POST	test_main	loginAdminMFA	Status: 200 (4 ms)
POST	test_main	loginMedicoA	Status: 200 (302 ms)
GET	test_main	3. SUCESSO: Acessar Rota Admin com Token Admin (MFA V	
GET	test_main	4. DEFESA BFLA: Acessar Rota Admin com Token de Médico	

GET http://127.0.0.1:8000/admin/dashboard

Show Request

HTTP/1.1 403 Forbidden

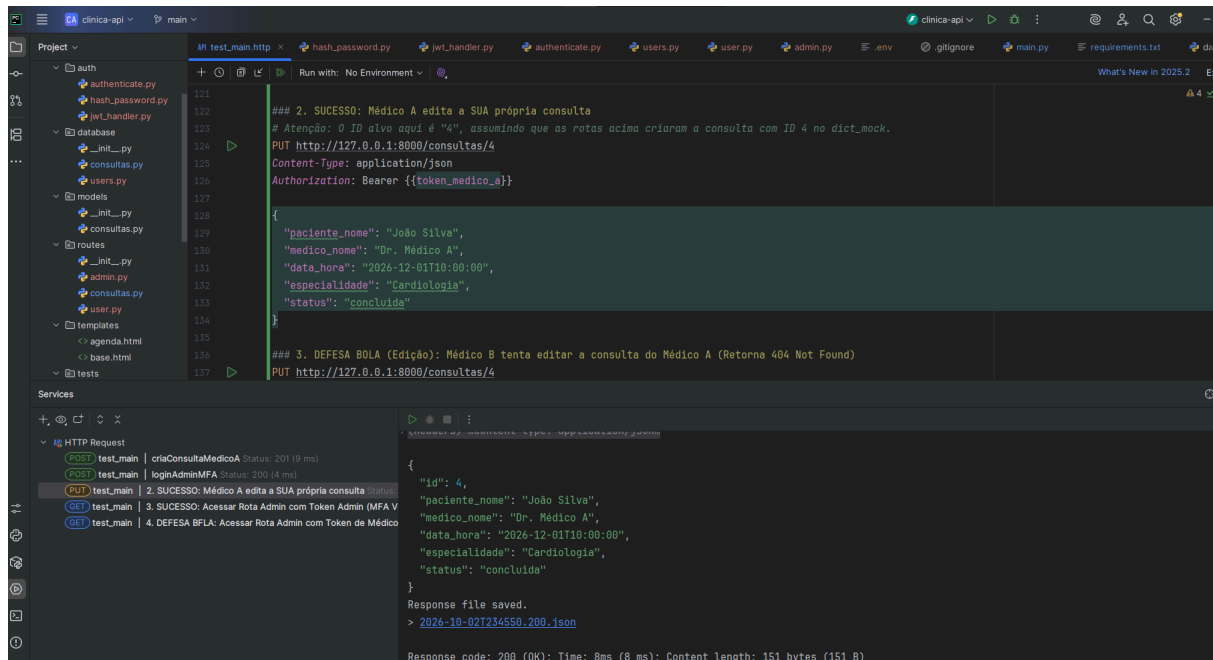
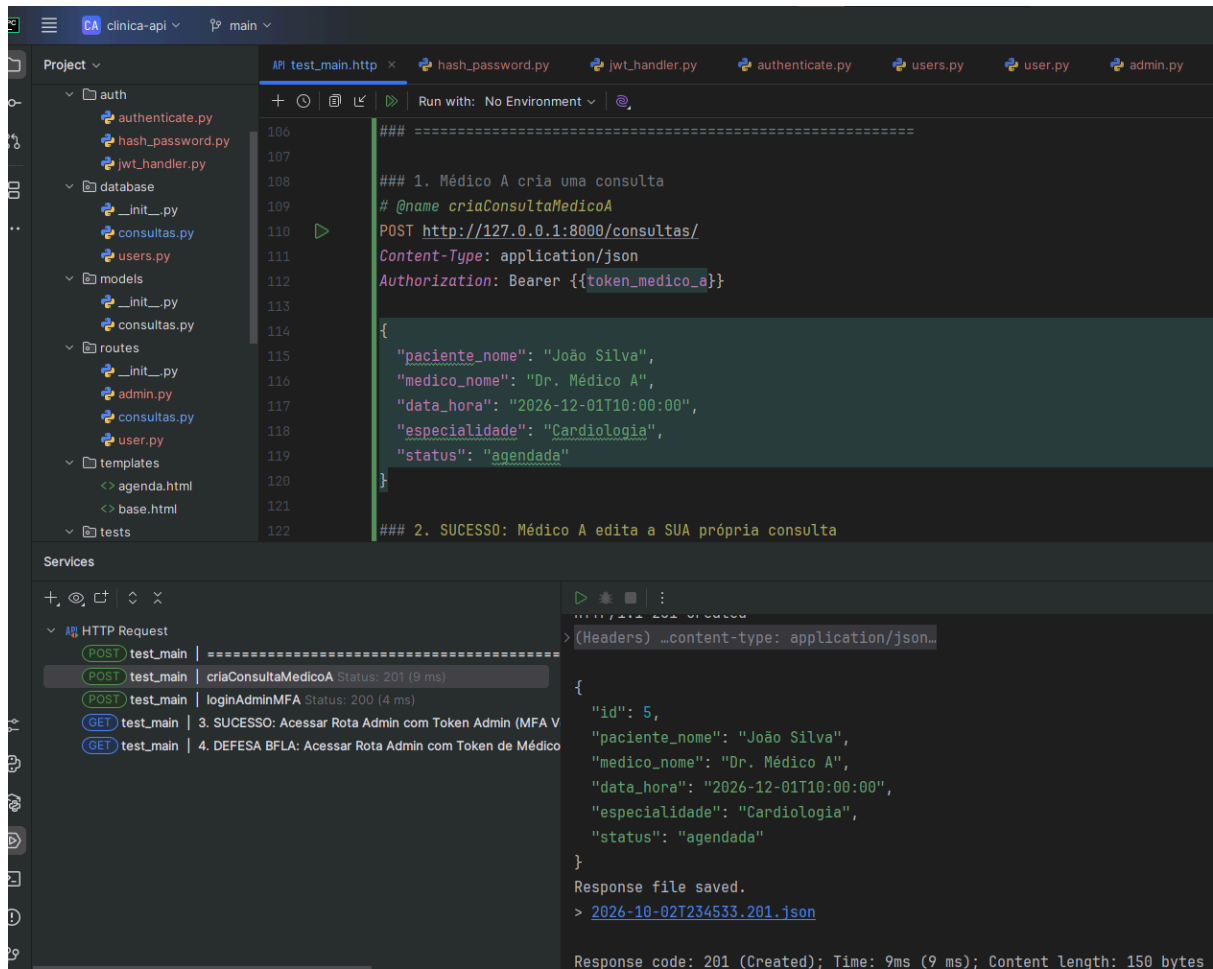
(Headers) ...content-type: application/json...

```
{
  "detail": "Acesso restrito a Administradores do sistema (BFLA Block)."
```

Response file saved.

> [2026-10-02T234517.403.json](#)

Response code: 403 (Forbidden); Time: 6ms (6 ms); Content length: 71 bytes (71 B)



```
133 "status": "concluida"
134
135
136
137 ## 3. DEFESA BOLA (Edição): Médico B tenta editar a consulta do Médico A (Retorna 404 Not Found)
138 PUT http://127.0.0.1:8000/consultas/4
139 Content-Type: application/json
140 Authorization: Bearer {{token_medico_b}}
141
142 {
143   "paciente_nome": "João Silva Wacked",
144   "medico_nome": "Dr. Médico B",
145   "data_hora": "2026-12-01T10:00:00",
146   "especialidade": "Cardiologia",
147   "status": "cancelada"
148 }
149
150 ## 4. DEFESA BOLA (Exclusão): Médico B tenta deletar a consulta do Médico A (Retorna 404 Not Found)
```

Services

HTTP Request

test_main | 1. criaConsultaMedicoA Status: 201 (9 ms)

test_main | 2. SUCESSO: Médico A edita a SUA própria consulta Status:

test_main | 3. DEFESA BOLA (Edição): Médico B tenta editar a consulta Status:

test_main | 3. SUCESSO: Acessar Rota Admin com Token Admin (MFA V

test_main | 4. DEFESA BOLA: Acessar Rota Admin com Token de Médico

PUT http://127.0.0.1:8000/consultas/4

Show Request

HTTP/1.1 404 Not Found

(Headers) _content-type: application/json

{

"detail": "Consulta não encontrada."

}

Response file saved.

> 2026-10-02T234605.404.json

Response code: 404 (Not Found); Time: 6ms (6 ms); Content length: 37 bytes (37 B)

```
147
148
149 ## 4. DEFESA BOLA (Exclusão): Médico B tenta deletar a consulta do Médico A (Retorna 404 Not Found)
150 DELETE http://127.0.0.1:8000/consultas/4
151 Authorization: Bearer {{token_medico_b}}
```

Services

HTTP Request

test_main | 1. criaConsultaMedicoA Status: 201 (9 ms)

test_main | 2. SUCESSO: Médico A edita a SUA própria consulta Status:

test_main | 3. DEFESA BOLA (Edição): Médico B tenta editar a consulta Status:

test_main | 3. SUCESSO: Acessar Rota Admin com Token Admin (MFA V

test_main | 4. DEFESA BOLA: Acessar Rota Admin com Token de Médico

test_main | 4. DEFESA BOLA (Exclusão): Médico B tenta deletar a co

DELETE http://127.0.0.1:8000/consultas/4

Show Request

HTTP/1.1 404 Not Found

(Headers) _content-type: application/json

{

"detail": "Consulta não encontrada."

}

Response file saved.

> 2026-10-02T234620.404.json

Response code: 404 (Not Found); Time: 4ms (4 ms); Content length: 37 bytes (37 B)

Relatório de Implementação - Exercício 7

Enunciado:

Selecione e implemente o fluxo OAuth 2.0 adequado ao cenário M2M, configurando escopos OAuth e claims JWT que diferenciem o acesso do laboratório parceiro do acesso de um profissional de saúde autenticado.

1. Objetivo

O Exercício 7 documenta a integração segura de um laboratório parceiro através da comunicação Máquina-a-Máquina (M2M) sem intervenção de usuário humano. O foco principal foi a adoção do fluxo OAuth 2.0 (Client Credentials Grant), restringindo o acesso unicamente à leitura de vagas

disponíveis, e utilizando Escopos (Scopes) como controle explícito para prevenir ataques de escalonamento de privilégios.

2. Arquitetura e Modularização

O ecossistema M2M exigiu adaptações pontuais, mantendo o isolamento arquitetural e a clareza de responsabilidades:

Módulo	Responsabilidade e Implementação
database/users.py	O dicionário em memória atuou como provedor de identidade (IdP), armazenando as credenciais (client_id e client_secret sob hash Bcrypt) e os escopos permitidos.
auth/authenticate.py	O módulo evoluiu para incorporar o Security Gate get_current_user_with_scopes, capaz de extrair a claim scope do JWT e forçar requisitos de acesso antes de executar o controlador.
routes/user.py	Um endpoint padronizado (POST /oauth/token) foi estabelecido para a negociação de tokens via grant_type="client_credentials".
routes/consultas.py	Criação do endpoint específico GET /consultas/vagas-laboratorio.

3. Controles de Segurança Aplicados (AppSec)

A segurança da integração focou em restringir os acessos e proteger dados médicos com base nos princípios fundamentais de segurança da informação:

3.1. Autenticação OAuth 2.0 (Client Credentials Grant) e Defesa M2M

- **Controle:** O token de acesso só é emitido se as credenciais do cliente forem validadas criptograficamente (Bcrypt).
- **Mitigação:** Evita ataques onde agentes não mapeados invadem rotas dedicadas à integração. O JWT gerado possui vida útil curta (60 minutos) e estampa o campo sub para fins de auditoria e rastreabilidade (Logs).

3.2. Autorização Baseada em Escopos (Scope-based Access Control)

- **Controle:** A rota M2M requisita explicitamente o escopo `appointments:read`.
- **Mitigação:** Ataca frontalmente falhas do tipo *Broken Access Control* (OWASP API1:2023 / BFLA). Mesmo que um invasor ou parceiro comprometa o token, o sistema negará (HTTP 403 Forbidden) qualquer tentativa de chamar rotas de criação (POST) ou exclusão (DELETE) que demandam o escopo `appointments:write`.

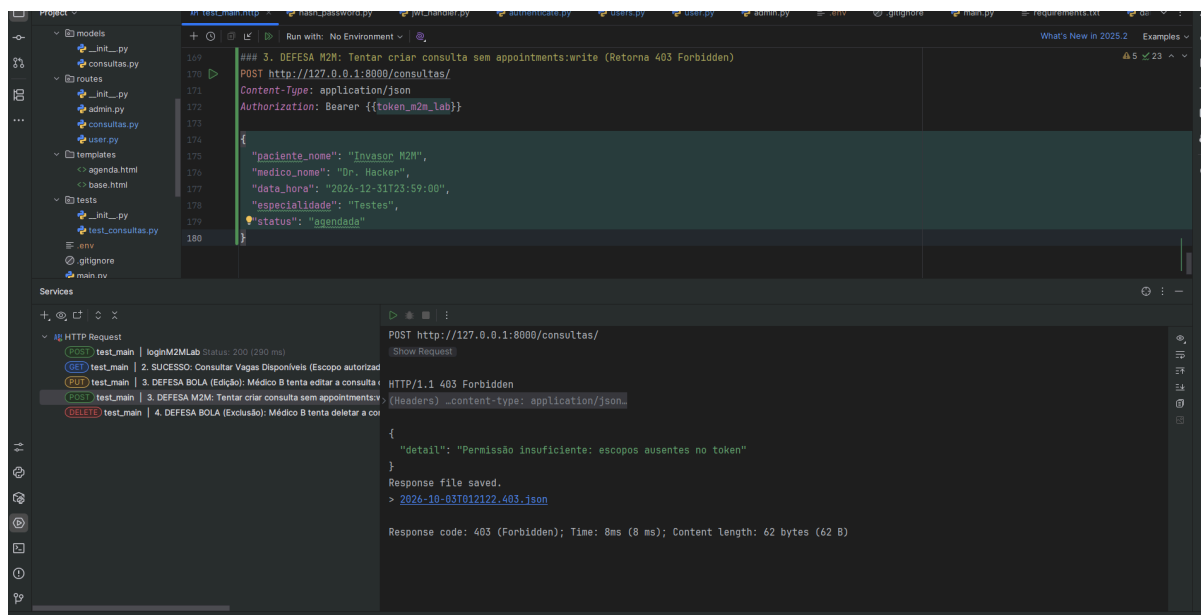
3.3. Egress Filtering Extremo (Proteção de PHI)

- **Controle:** O endpoint `/consultas/vagas-laboratorio` foi blindado para iterar os dados em memória e montar manualmente o pacote de resposta.
- **Mitigação:** O *Egress Filtering* rigoroso oculta permanentemente campos como `paciente_nome` e `anotacoes_internas`. Assim, a API cumpre o Princípio do Privilégio Mínimo no acesso à informação, preservando a confidencialidade exigida (Tríade CIA) e barrando a exposição acidental de dados sensíveis (PHI - Protected Health Information).

4. Evidências de Automação e Testes

Os testes automatizados e operacionais confirmaram a eficiência das defesas implementadas:

- **Testes Automatizados (Pytest):** A suíte alcançou **100% de aprovação** comprovando a emissão do token e o bloqueio estrito em rotas restritas.
- **Testes HTTP Manuais (REST Client):**
 - Geração com sucesso (HTTP 200 OK) do JWT do `lab_parceiro_01` com escopo de leitura.
 - Consumo com sucesso (HTTP 200 OK) da lista de horários.
 - Bloqueio imediato (HTTP 403 Forbidden) ao forjar uma chamada de inserção de consultas (POST `/consultas/`) com o token de escopo limitado ("*Permissão insuficiente: escopos ausentes no token*").



Relatório de Auditoria de Segurança - Exercício 8

Enunciado:

Analise o código da aplicação e identifique pelo menos três padrões vulneráveis de categorias distintas do OWASP Top 10, incluindo uma instância de BOLA (Broken Object Level Authorization).

1. Análise Estática e Identificação de Vulnerabilidades

Abaixo, detalhamos três vulnerabilidades críticas identificadas nos artefatos da API antes da implementação do Security Gate e validações de propriedade.

VULNERABILIDADE 1: OWASP API1:2023 - Broken Object Level Authorization (BOLA / IDOR)

O BOLA ocorre quando a aplicação falha em validar se o usuário autenticado possui os direitos de propriedade ou acesso sobre um objeto específico solicitado via identificador (ID). Este é caracterizado como um ataque de escalonamento horizontal.

Trecho de Código Vulnerável (Antes do Hardening):

```
@router.get("/{id}", response_model=ConsultaResponse)
async def obter_consulta_vulneravel(
    id: int = Path(...),
    current_user: dict = Depends(get_current_user)
):
    consulta = consultas_db.get(id)
    if not consulta:
```

```

        raise
    HTTPException(status_code=status.HTTP_404_NOT_FOUND)
    # VULNERABILIDADE: Retorna a consulta sem checar o
    "ownership"
    return consulta

```

- **Explicação Técnica da Falha:** A rota valida se o usuário está autenticado (Depends(get_current_user)), mas confia cegamente no parâmetro id fornecido na URL. Não há nenhuma instrução condicional que compare o identificador do usuário logado (ex: current_user["sub"]) com o ID do dono associado ao registro no banco de dados.
- **Cenário Exploratório:** O Paciente A (ou Médico B) autentica-se legitimamente na API e intercepta sua própria requisição. Ele manipula o identificador na URI de GET /consultas/1 para GET /consultas/2. O servidor processa a requisição e devolve os dados da consulta pertencente ao Paciente B.
- **Impacto no Negócio:** Violação direta da confidencialidade (Tríade CIA). A exposição não autorizada de dados sensíveis de saúde (PHI) fere gravemente o Art. 5º, II da LGPD, sujeitando a clínica a multas severas e danos reputacionais irreparáveis.

VULNERABILIDADE 2: OWASP API3:2023 - Broken Object Property Level Authorization (BOPLA / Mass Assignment)

O BOPLA (especificamente o vetor de Mass Assignment) ocorre quando o backend aceita dados do cliente e os vincula (hidrata) diretamente aos modelos de dados sem uma filtragem estrita de propriedades, permitindo a adulteração de atributos internos.

Trecho de Código Vulnerável (Antes do Hardening):

```

class ConsultaCreate(BaseModel):
    """
    Modelo de Ingress (Entrada).
    Garante que apenas os dados definidos sejam aceitos e
    validados.
    """
    # VULNERABILIDADE: O modelo aceita e confia em campos de
    controle do sistema
    paciente_nome: str = Field(..., min_length=2,
max_length=100)
    medico_nome: str = Field(..., min_length=2, max_length=100)
    data_hora: str = Field(..., min_length=16, max_length=25)

```

```

    especialidade: str = Field(..., min_length=2,
max_length=50)
    status: str = Field(..., min_length=2, max_length=20) #
Campo crítico de estado liberado para o input do usuário
    # Ausência de bloqueio para campos não mapeados
(extra='forbid')

```

- **Explicação Técnica da Falha:** O modelo Pydantic atuando no *Ingress* da aplicação expõe a propriedade `status` para modificação direta via payload JSON. Além disso, a falta de configurações de restrição permite que propriedades extras enviadas no corpo da requisição sejam mapeadas inadvertidamente pelo ORM/dicionário.
- **Cenário Exploratório:** Um atacante (ou usuário mal-intencionado) envia uma requisição POST `/consultas/legado` ou de atualização, injetando o par `"status": "concluida"` ou propriedades internas não documentadas no corpo do JSON. A API aceita o payload e sobrescreve o estado do agendamento.
- **Impacto no Negócio:** Quebra da Integridade dos dados. Pacientes podem forçar o encerramento de consultas sem pagamento prévio, cancelar consultas de terceiros ou fraudar fluxos operacionais, gerando falhas agudas no faturamento e na gestão de horários da clínica.

VULNERABILIDADE 3: OWASP API5:2023 - Broken Function Level Authorization (BFLA)

O BFLA ocorre quando endpoints sensíveis ou administrativos não implementam verificações robustas de autorização baseadas em papéis (RBAC), resultando em escalonamento vertical de privilégios.

Trecho de Código Vulnerável (Antes do Hardening):

```

@router.get("/dashboard")
async def get_admin_dashboard(claims: dict = Depends(require_admin)):
    """Rota protegida para Administradores."""
    # VULNERABILIDADE: Verifica o token, mas ignora o papel
    (Role) do usuário
    return {
        "message": "Bem-vindo ao Painel de Controle (Admin).",
        "admin_id": claims.get("sub")
    }

```

- **Explicação Técnica da Falha:** A rota administrativa utiliza a dependência `get_current_user`, que apenas verifica a validade criptográfica do token JWT (Autenticação). A aplicação falha ao omitir o *Security Gate* de

autorização que garantiria que a *claim* role no payload do JWT fosse estritamente igual a "admin".

- **Cenário Exploratório:** Um paciente ou médico autenticado inspeciona o tráfego e descobre o endpoint `/admin/dashboard`. Ele anexa seu JWT padrão no cabeçalho `Authorization` e dispara a requisição. O servidor não avalia as permissões e devolve HTTP 200 OK, expondo o dashboard gerencial.
- **Impacto no Negócio:** Elevação vertical de privilégios. O atacante ganha acesso irrestrito às funções críticas de administração, podendo extrair bases completas de pacientes, apagar registros de auditoria ou alterar configurações sistêmicas, resultando em comprometimento total da infraestrutura lógica.

2. Quadro Resumo e Matriz de Risco das Vulnerabilidades Identificadas

ID	Categoria OWASP API Top 10	Nome da Vulnerabilidade	Rota Afetada	Nível de Risco	Estratégia de Mitigação Aplicada
V1	API1:2023	Broken Object Level Authorization (BOLA)	GET /consultas/{id} PUT /consultas/{id}	Crítico	Implementação de validação de <i>ownership</i> no backend (<code>medico_id == current_user["sub"]</code>). Retorno estratégico de HTTP 404 Not Found em vez de 403 para evitar vazamento de existência do recurso (Enumeração).
V2	API3:2023	Broken Object Property Level Authorization (BOPLA)	POST /consultas/	Alto	Desacoplamento de modelos via Pydantic (Ingress vs Egress Filtering). Remoção de

ID	Categoria OWASP API Top 10	Nome da Vulnerabilidade	Rota Afetada	Nível de Risco	Estratégia de Mitigação Aplicada
			POST /consultas/legado		campos de estado (status, anotacoes_internas) do ConsultaCreate. Definição segura do estado interno diretamente no controlador.
V3	API5:2023	Broken Function Level Authorization (BFLA)	GET /admin/dashboard	Crítico	Criação de injeção de dependência restrita (require_admin) garantindo RBAC (claims.get("role") == "admin"). Implementação de bloqueios explícitos com HTTP 403 Forbidden e exigência de MFA para ações administrativas.

3. ORIENTAÇÃO DE MANUTENÇÃO DE EVIDÊNCIAS DE TESTE

Para garantir a erradicação dos riscos OWASP Top 10 e prevenir regressões sistêmicas, a suíte de testes do projeto foi arquitetada seguindo as premissas do *Test-Driven Security*.

A. Transição do Estado Vulnerável para o Estado Mitigado (Testes de Abuso):

- Durante a fase de desenvolvimento, os testes inicialmente simulavam o sucesso de ataques de privilégio. Após a aplicação do *hardening*, esses cenários foram mantidos no pytest (test_consultas.py) e no arquivo HTTP Client (test_main.http), mas agora com asserções (assertions) que

validam a **rejeição** da requisição pelo sistema, comprovando o isolamento da ameaça.

B. Evidências no Arquivo `test_main.http`:

- **Mitigação de BOLA:** O bloco # DEFESA BOLA (Edição) demonstra o Médico B tentando alterar a consulta ID 4 (pertencente ao Médico A). A evidência de segurança reside no fato do endpoint agora retornar 404 Not Found, ocultando a existência do recurso de usuários não proprietários (prevenindo enumeração).
- **Mitigação de M2M/Scopes:** A manutenção da chamada # DEFESA M2M: Tentar criar consulta sem appointments:write valida o isolamento de ambientes e garante que um token M2M Client Credentials com escopo restrito (read) seja bloqueado com 403 Forbidden ao tentar manipular dados.

C. Evidências Automatizadas via pytest (`test_consultas.py`):

- **Mitigação BFLA:** O teste `test_bloqueio_usuario_nao_admin_em_rota_restrita_admin` atesta que a injeção da dependência de validação de *roles* está operante. O teste força o login do "Médico A", captura o JWT e realiza um GET em `/admin/dashboard`, validando se o HTTP Status é rigorosamente 403 e atestando que a vulnerabilidade de escalonamento vertical foi saneada.
- **Mitigação BOPLA (Egress Filtering):** O teste `test_garantir_nao_vazamento_anotacoes_internas_json` garante dinamicamente que propriedades como `anotacoes_internas` não sejam enviadas de volta ao cliente, atestando o funcionamento dos Modelos de Resposta Pydantic (`ConsultaResponse`) na mitigação da exposição excessiva de dados.

Relatório de Implementação - Exercício 9

Enunciado:

Corrija as vulnerabilidades identificadas no Exercício 8, implementando validação whitelist e regex, middleware JWT centralizado com verificação de ownership, Pydantic com `extra='forbid'` e output encoding via auto-escape do Jinja2. Identifique, além disso, pelo menos um endpoint da aplicação que não foi citado explicitamente no Exercício 8 mas compartilha o mesmo padrão de vulnerabilidade, e aplique a mesma correção a ele.

1. Objetivo

Este relatório documenta a fase de mitigação e remediação estrutural das vulnerabilidades identificadas durante a auditoria de segurança (Exercício 8). O Exercício 9 teve como foco a aplicação prática dos princípios de Secure by Design e Defense in Depth (Defesa em Profundidade) na API, erradicando falhas críticas de Autorização (BOLA e BOPLA), injeções de dados e gerenciamento indevido de inventário (Zombie APIs), consolidando uma arquitetura resiliente e segura para o ecossistema médico.

2. Controles de Segurança Aplicados (Remediação)

2.1. Mitigação de BOLA (Broken Object Level Authorization) e Ownership

A vulnerabilidade mais crítica identificada permitia que um médico acessasse, editasse ou excluísse consultas de terceiros.

- **Controle Implementado:** Foi criada a injeção de dependência centralizada `get_owned_consulta` e aplicada aos endpoints sensíveis (GET `/id`, PUT `/id`, DELETE `/id` e visualização web). O backend agora extrai obrigatoriamente a identidade do usuário logado via token JWT (`current_user["sub"]`) e cruza com o `medico_id` registrado no banco de dados.
- **Prevenção de Enumeração:** Em vez de retornar um erro explícito de permissão (403 Forbidden), o *Security Gate* foi projetado para lançar a exceção 404 Not Found. Isso mitiga vulnerabilidades auxiliares de *Information Disclosure*, impedindo que um atacante mapeie IDs válidos varrendo o sistema.

2.2. Proteção contra Mass Assignment / BOPLA (Broken Object Property Level Authorization)

Os modelos de entrada possuíam tipagem forte, mas confiavam cegamente no payload, permitindo a injeção de campos não mapeados (como tentar forçar a propriedade `is_admin = true`).

- **Controle Implementado:** O modelo de entrada Pydantic v2 (`ConsultaCreate`) foi atualizado com a configuração `model_config = ConfigDict(extra='forbid')`.
- **Mitigação:** Qualquer pacote JSON que contenha parâmetros não documentados na fronteira da API é sumariamente rejeitado pelo FastAPI, retornando um erro 422 Unprocessable Content e bloqueando a requisição antes mesmo de atingir os controladores lógicos.

2.3. Validação Ativa (Allow-list) e Sanitização contra Injeções (SQLi / XSS)

Dois vetores de injeção foram tratados nos sistemas de busca e visualização.

- **Regex Whitelisting (Busca):** A nova rota de busca (GET `/busca`) recebeu validação paramétrica estrita baseada em Regex (`^[a-zA-Z0-9\s]+$`).

Caracteres frequentemente utilizados em injeções (como aspas simples ', ;, -- ou <) são bloqueados com 400 Bad Request.

- **Auto-escape Jinja2 (XSS):** O filtro `| safe` foi removido do arquivo `agenda.html`. Ao forçar a renderização com a sintaxe padrão do Jinja2, a aplicação passou a neutralizar passivamente (escapar) qualquer script malicioso (Ex: `<script>`) que venha a ser salvo no banco de dados, transformando-o em texto inofensivo no painel da recepção.

2.4. Deprovisionamento Seguro (Mitigação de Zombie API)

A análise de inventário detectou a manutenção indevida da rota legada (POST /legado), que criava um *bypass* (porta dos fundos) por não exigir token JWT nem o Pydantic restrito.

- **Controle Implementado:** Seguindo o princípio de gestão de ciclo de vida seguro (OWASP API9:2023 - Improper Inventory Management), o endpoint foi desativado/comentado no código-fonte, fechando a brecha. O histórico da remoção foi mantido para fins de auditoria de código.

3. Defesa em Profundidade (Estudo de Caso: Rota de Busca)

A rota /busca consolidou a aplicação de múltiplos pilares de segurança sobrepostos na mesma ação:

1. **Authentication (AuthN):** O acesso anônimo foi bloqueado; a rota exige obrigatoriamente um token JWT válido para responder.
2. **Input Validation:** O parâmetro passa por validação explícita de caracteres (Regex).
3. **Authorization (BOLA / RBAC):** Após buscar os dados, a lógica aplica *Ownership*, devolvendo ao Médico apenas resultados vinculados ao seu próprio `medico_id`, enquanto a Recepcionista tem visibilidade global.

4. Evidências de Automação e Testes (Quality Gates)

Para atestar que as refatorações arquiteturais não quebraram as regras de negócio e bloqueiam efetivamente os ataques, duas baterias foram consolidadas:

- **Testes Automatizados (Pytest):** A suíte de regressão rodou 100% no verde. Foram criados testes dedicados para o Exercício 9, assertando o bloqueio estrutural:
 - `test_defesa_bola_consulta_terceiros`: Valida o falso 404.
 - `test_defesa_bopla_campos_extras`: Valida o bloqueio 422.
 - `test_defesa_regex_busca_maliciosa`: Valida o bloqueio 400.
 - Validação de XSS atestando a presença de `<script>` escapado na renderização HTML.
- **Testes de Intrusão Simulada (REST Client - test_main.http):** Foram executadas requisições simulando um ator malicioso interceptando a comunicação e modificando payloads e chaves de autorização. Todos

os acessos indevidos e inserções anômalas resultaram em recusa categórica da aplicação.

```
...
├── _init_.py
├── consultas.py
├── routes
│   ├── _init_.py
│   ├── admin.py
│   ├── consultas.py
│   └── user.py
├── templates
│   ├── agenda.html
│   ├── base.html
│   └── ...
├── tests
│   ├── _init_.py
│   └── test_consultas.py
├── .env
├── .gitignore
├── main.py
├── requirements.txt
└── test_main.http

External Libraries

Terminal  Git Bash  Local  Local (2)
collected 12 items

tests/test_consultas.py .....

===== 12 passed in 5.03s =====

(.venv) PS C:\Users\matheus.fsantos\PycharmProjects\clinica-api> pytest tests/
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.1.1, pluggy-1.6.0
rootdir: c:\Users\matheus.fsantos\PycharmProjects\clinica-api
plugins: anyio-4.15.1
collected 14 items

tests/test_consultas.py .....

===== 14 passed in 5.75s =====

(.venv) PS C:\Users\matheus.fsantos\PycharmProjects\clinica-api> 
```

[illegible]

The image shows a VS Code editor with a REST client file named `rest_client.http` and its corresponding test results in the Output panel.

rest_client.http content:

```
197  
198 ### 2. DEFESA BOLA (DELETE): Médico B tenta deletar consulta do Médico A (Retorna 404 Not Found)  
199 DELETE http://127.0.0.1:8000/consultas/1  
200 Authorization: Bearer {{token_medico_b}}  
201  
202  
203 ### 3. DEFESA BOPLA (Mass Assignment): Criar consulta enviando campos extras não documentados (Retorna 422 Unprocessable Content)  
204 POST http://127.0.0.1:8000/consultas/  
205 Content-Type: application/json  
Authorization: Bearer {{token_medico_a}}
```

Test Results (Output Panel):

✓ Stopped. 30 tests passed 30 / 55 tests, 3 sec 180 ms

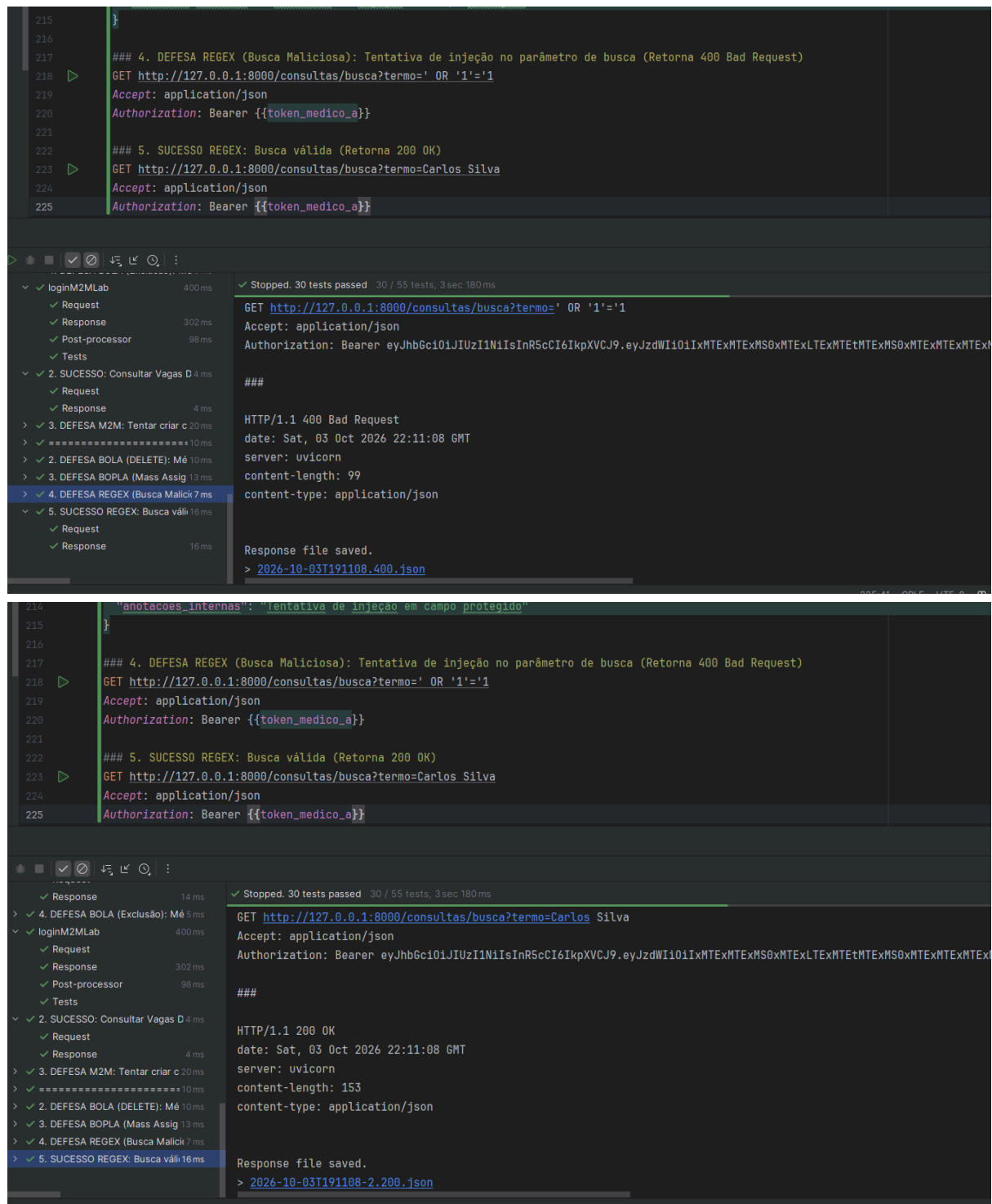
- ✓ Request
- ✓ Response 12 ms
- ✓ 3. DEFESA BOLA (Edição): Médico B tenta deletar consulta do Médico A (Retorna 404 Not Found) 14 ms
 - ✓ Request
 - ✓ Response 14 ms
- > 4. DEFESA BOLA (Exclusão): Médico B tenta deletar consulta do Médico A (Retorna 404 Not Found) 5 ms
- ✓ loginM2MLab 400 ms
 - ✓ Request
 - ✓ Response 302 ms
 - ✓ Post-processor 98 ms
 - ✓ Tests
- ✓ 2. SUCESSO: Consultar Vagas Disponíveis 4 ms
 - ✓ Request
 - ✓ Response 4 ms
- > 3. DEFESA M2M: Tentar criar consulta 20 ms
- > ===== 10 ms
- > 2. DEFESA BOLA (DELETE): Médico B tenta deletar consulta do Médico A (Retorna 404 Not Found) 10 ms
 - ✓ 3. DEFESA BOPLA (Mass Assignment): Criar consulta enviando campos extras não documentados (Retorna 422 Unprocessable Content) 13 ms
 - ✓ 4. DEFESA REGEX (Busca Malick) 7 ms
 - ✓ 5. SUCESSO REGEX: Busca válida 16 ms
 - ✓ Request

Test Details for 2. DEFESA BOLA (DELETE):

Response file saved.
> 2026-10-03T191108-1.404.json

Response code: 404 (Not Found); Time: 10ms (10 ms); Content length: 37 bytes (37 B)

```
199 DELETE http://127.0.0.1:8000/consultas/1
200 Authorization: Bearer {{token_medico_b}}
201
202 ### 3. DEFESA BOPLA (Mass Assignment): Criar consulta enviando campos extras não documentados (Retorna 422 Unprocessable Content)
203 POST http://127.0.0.1:8000/consultas/
204 Content-Type: application/json
205 Authorization: Bearer {{token_medico_a}}
206
207 {
208   "paciente_nome": "Invasor Malicioso",
209   "medico_nome": "Dr. Hacker",
210   "data_hora": "2026-12-31T23:59:00",
211   "especialidade": "Exploitation",
212   "status": "agendada",
213   "is_admin": true,
214   "anotacoes_internas": "Tentativa de injeção em campo protegido"
215 }
216
217 HTTP/1.1 422 Unprocessable Content
218 date: Sat, 03 Oct 2026 22:11:08 GMT
219 server: uvicorn
220 content-length: 271
221 content-type: application/json
222
223 Response file saved.
```



Relatório de Implementação - Exercício 10

Enunciado:

Configure CORS com allowlist explícita, implemente os cabeçalhos de segurança HSTS, X-Frame-Options e X-Content-Type-Options, e implemente rate limiting diferenciado para o endpoint de login.

1. Objetivo

O objetivo desta etapa foi estabelecer a "Defesa em Profundidade" (Defense in Depth) mitigando vetores de ataque automatizados, abusos de consumo de recursos e interações maliciosas via navegador utilizando Middlewares globais e Rate Limiting.

2. Arquitetura e Controles de Rede

Para garantir que o tráfego hostil seja interceptado antes mesmo de atingir a lógica de negócios ou as rotas de acesso a dados, as proteções foram estabelecidas na borda da aplicação (Security Gates de Rede). O FastAPI foi configurado para processar toda requisição de entrada através de uma esteira de middlewares de validação (CORS e Headers) e de uma verificação granular de cota de consumo baseada em IP (Slowapi).

3. Controles de Segurança Aplicados

As seguintes camadas de defesa foram implementadas para proteger os recursos expostos:

3.1. CORS Rigoroso (Cross-Origin Resource Sharing)

Foi configurado o CORSMiddleware abandonando a prática insegura do uso de *wildcard* (*).

- **Controle:** Implementação de uma *Allowlist* explícita (origins = ["http://localhost:3000", ...]).
- **Mitigação:** Previne o roubo de dados por scripts maliciosos hospedados em domínios de terceiros (*Information Disclosure* / Ameaças de Client-side), garantindo que navegadores modernos bloqueiem respostas caso a origem da requisição não esteja autorizada.

3.2. Cabeçalhos de Segurança HTTP (Security Headers)

Foi desenvolvido um middleware assíncrono que injeta cabeçalhos de segurança obrigatórios em **todas** as respostas da API.

- **Controle:** Injeção padronizada de três headers críticos.
- **Mitigação:**
 - Strict-Transport-Security (HSTS): Força o tráfego exclusivo em HTTPS, protegendo contra *Downgrade Attacks* e interceptação Man-in-the-Middle.
 - X-Frame-Options: DENY: Bloqueia a renderização da API em iframes, erradicando a viabilidade de ataques de *Clickjacking*.
 - X-Content-Type-Options: nosniff: Impede que o navegador infira o *MIME type* do conteúdo, mitigando a execução acidental ou induzida de scripts maliciosos mascarados.

3.3. Rate Limiting Granular e Diferenciado

A biblioteca slowapi foi integrada para proteger os endpoints contra abuso contínuo, utilizando o IP de origem como chave de identificação.

- **Controle Estrito (/user/login):** Limite estabelecido em **5 requisições por minuto**.
- **Controle Permissivo (/consultas/):** Limite estabelecido em **60 requisições por minuto**.
- **Mitigação:** A granularidade protege diretamente o recurso de autenticação contra ataques de Força Bruta (*Brute Force*) e *Credential Stuffing* (Spoofing). Simultaneamente, preserva a disponibilidade do catálogo principal contra exaustão de recursos (*Denial of Service*), sem impactar negativamente a experiência de usuários legítimos. O bloqueio resulta na recusa automática da requisição via status HTTP 429 Too Many Requests.

4. Atualização do Modelo de Ameaças (STRIDE)

A implementação deste exercício consolida o mapeamento do framework STRIDE para a infraestrutura de borda:

Componente da API	Categoria STRIDE	Ameaça Específica Identificada	Mitigação Implementada
Middleware globais	Information Disclosure	Scripts em sites não autorizados tentando ler dados (PHI) de pacientes.	CORS: Aplicação de Allowlist explícita.
Middleware globais	Elevation of Privilege / Spoofing	Sobreposição de frames (Clickjacking) ou Downgrade de tráfego.	Security Headers: HSTS, X-Frame-Options (DENY), X-Content-Type-Options (nosniff).

Componente da API	Categoria STRIDE	Ameaça Específica Identificada	Mitigação Implementada
Rota /user/login	Spoofing	Força bruta para descobrir senhas e acessar contas médicas/admin.	Rate Limit (Estrito): Máximo de 5 req/min.
Rota /consultas/	Denial of Service	Robôs exaurindo a capacidade de processamento do servidor FastAPI.	Rate Limit (Permissivo): Máximo de 60 req/min.

5. Evidências de Automação e Testes (Quality Gates)

A eficácia dos controles de borda foi atestada por meio da automação de testes sob a premissa de *Test-Driven Security*:

- Testes Automatizados (Pytest):** Foram escritos testes específicos para a validação dos Headers e do bloqueio do CORS. O teste `test_rate_limiting_estouro_login` forçou uma sequência iterativa no servidor, confirmando que a 6ª requisição é devidamente barrada com status 429. Uma fixture de reset global foi incorporada no Pytest para isolar a memória do limitador, mantendo o histórico de execução 100% verde sem colisão entre cenários.
- Testes de Intrusão Simulada (REST Client - test_main.http):** A bateria de testes operacionais manuais evidenciou o funcionamento no mundo real.
 - Sucesso no fornecimento dos cabeçalhos extras para origens autorizadas.
 - Omissão proposital do header `Access-Control-Allow-Origin` em requisições simuladas por atacantes (bloqueio de navegador).
 - Retorno do payload JSON contendo `{"error": "Rate limit exceeded: 5 per 1 minute"}` seguido de log 429 Too Many Requests.

```
..._init_.py
consultas.py
routes
  _init_.py
  admin.py
  consultas.py
  user.py
templates
  agenda.html
  base.html

319 | assert response.json() == {'detail': 'consultas não encontradas.'}
320 |
321 | def test_validar_cors_origem_autorizada():...
322 |
323 | def test_validar_presenca_security_headers():...
324 |
325 | def test_rate_limiting_estouro_login():...

Terminal
Git Bash x Local x Local (2) x + v

===== 18 passed, 2 warnings in 7.67s =====
(.venv) PS C:\Users\matheus.fsantos\PycharmProjects\clinica-api> pytest tests/
platform win32 -- Python 3.14.2, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\matheus.fsantos\PycharmProjects\clinica-api
plugins: anyio-4.15.1
collected 18 items

tests\test_consultas.py .....

===== warnings summary =====
..venv\lib\site-packages\slowapi\extension.py:720
..venv\lib\site-packages\slowapi\extension.py:720
C:\Users\matheus.fsantos\PycharmProjects\clinica-api\..venv\lib\site-packages\slowapi\extension.py:720: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal in Python 3.16; use inspect.iscoroutinefunction() instead
  if asyncio.iscoroutinefunction(func):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 18 passed, 2 warnings in 7.83s =====
(.venv) PS C:\Users\matheus.fsantos\PycharmProjects\clinica-api>
```

consultas.py

user.py

templates

agenda.html

base.html

tests

init.py

test_consultas.py

.env

230

231

232

233

234

235

236

```
### Teste 1: CORS Origem Autorizada
# 0 Header 'Access-Control-Allow-Origin' estará presente na resposta HTTP 200 OK
GET http://127.0.0.1:8000/
Origin: http://localhost:3000
Accept: application/json
```

Services

loginM2MLab

Request

Response

Post-processor

Tests

2. SUCESSO: Consultar Vagas Dispon

3. DEFESA M2M: Tentar criar consu

===== 9 ms

Request

Response

2. DEFESA BOLA (DELETE): Médico

Request

Response

3. DEFESA BOPLA (Mass Assignment)

4. DEFESA REGEX (Busca Maliciosa)

Request

Response

5. SUCESSO REGEX: Busca válida (R

===== 19 ms

Teste 2: CORS Origem NÃO Autoriz

Teste 3: Validação de Security Head

Teste 4: Sequência de 5 POSTs no

Request

Response

Teste 5: Requisição em GET /consult

519 ms

442 ms

77 ms

6 ms

14 ms

9 ms

9 ms

13 ms

13 ms

6 ms

14 ms

14 ms

14 ms

19 ms

10 ms

18 ms

318 ms

9 ms

Stopped: 35 tests passed 35 / 65 tests, 3sec 253 ms

GET http://127.0.0.1:8000

Origin: http://localhost:3000

Accept: application/json

###

HTTP/1.1 200 OK

date: Sat, 03 Oct 2026 23:32:48 GMT

server: uvicorn

content-length: 58

content-type: application/json

access-control-allow-credentials: true

access-control-allow-origin: http://localhost:3000

vary: Origin

strict-transport-security: max-age=31536000; includeSubDomains

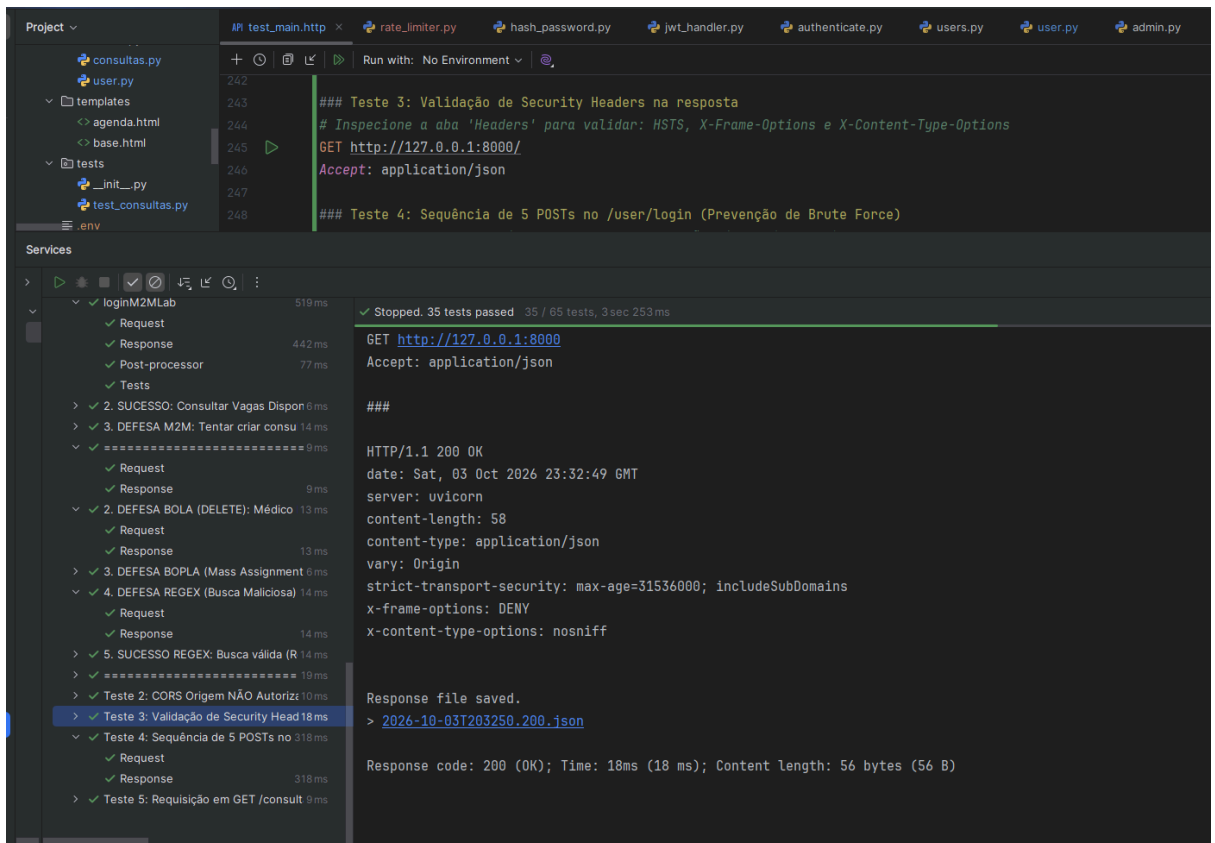
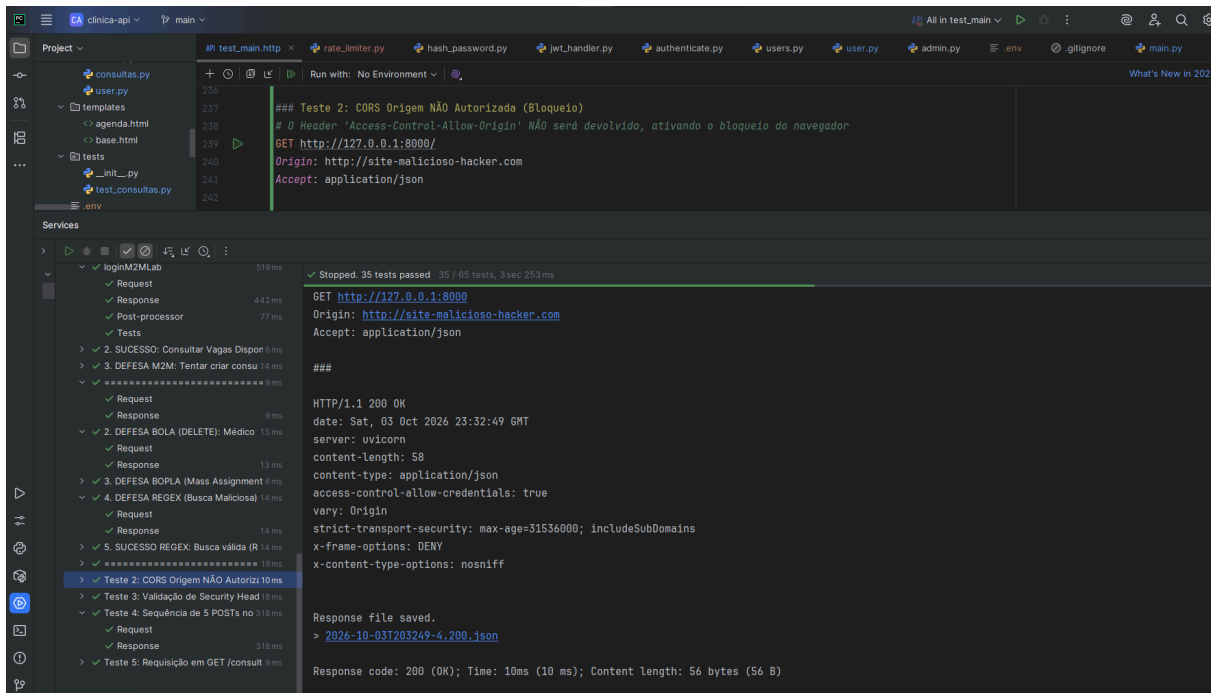
x-frame-options: DENY

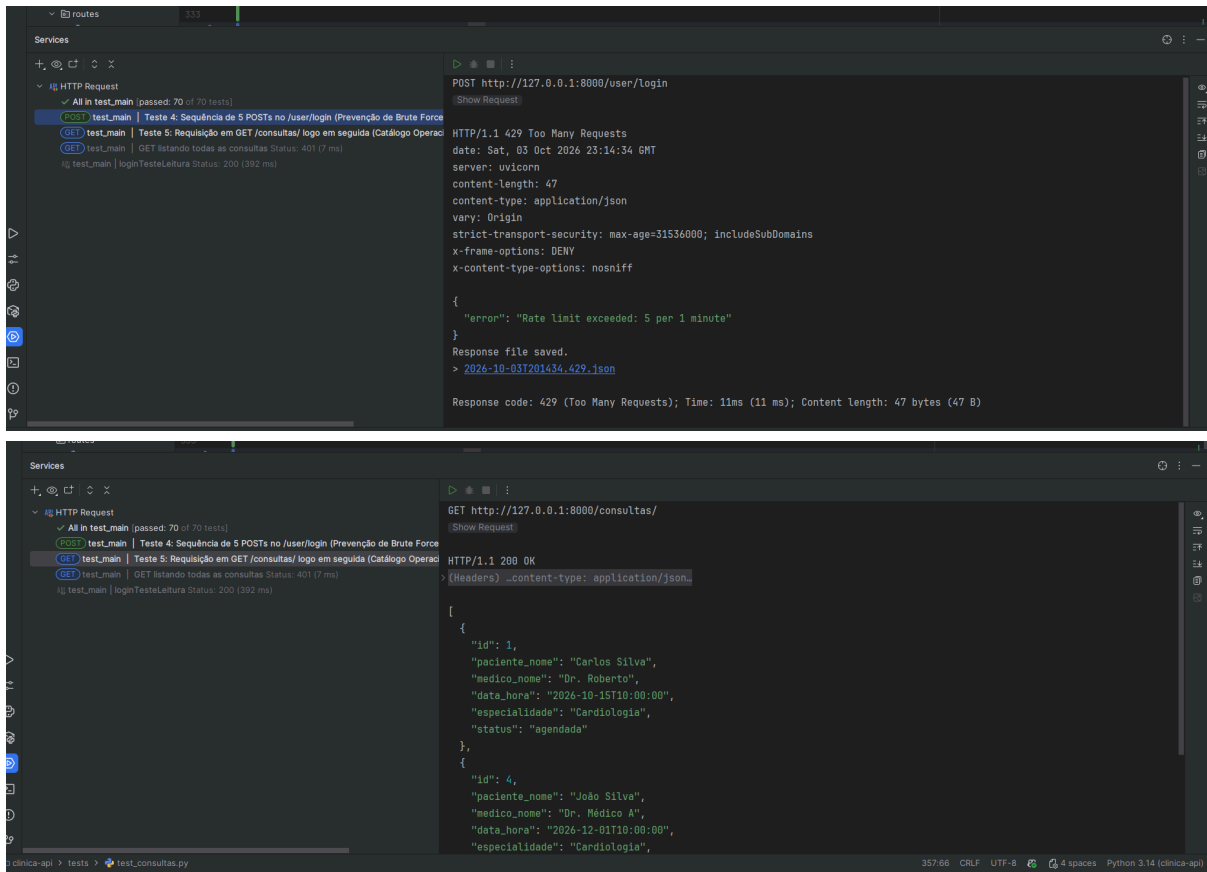
x-content-type-options: nosniff

Response file saved.

> 2026-10-03T203249-3.200.json

Response code: 200 (OK); Time: 19ms (19 ms); Content Length: 56 bytes (56 B)





Relatório de Implementação - Exercício 11

Enunciado:

Migre a camada de persistência para SQLAlchemy com queries parametrizadas, integre sessões via injeção de dependência e gerencie credenciais de banco via BaseSettings e arquivo .env.

1. Objetivo

O Exercício 11 teve como foco a substituição dos mocks em memória (dicionários) por um banco de dados relacional (SQLite) utilizando o SQLAlchemy (ORM). O objetivo central foi implementar operações de CRUD de forma segura, garantindo a proteção contra injeções de banco de dados (SQL Injection), o gerenciamento seguro de credenciais de ambiente e o isolamento completo da suíte de testes automatizados.

2. Arquitetura de Persistência e Configurações

Para garantir que a integração com o banco de dados escalasse de forma segura e sustentável, o código foi refatorado respeitando o isolamento de responsabilidades:

- **config/settings.py & .env:** Implementação da biblioteca pydantic-settings para externalizar segredos (chaves JWT, URLs de banco) do código-fonte, garantindo que credenciais reais nunca sejam versionadas no repositório.
- **database/connection.py:** Centralização da criação do motor do banco (create_engine) e da gerência de sessões, utilizando Injeção de Dependência (get_session) no FastAPI para gerenciar o ciclo de vida das conexões.
- **Lifespan Events:** Atualização do ciclo de vida da aplicação no main.py utilizando @asynccontextmanager async def lifespan, substituindo eventos depreciados e garantindo a correta inicialização das tabelas e injeção de dados simulados (Seeding) antes de a API aceitar requisições.

3. Controles de Segurança Aplicados

A transição para um banco de dados real introduz novos vetores de ataque. Para neutralizá-los, os seguintes controles arquiteturais foram aplicados:

3.1. Prevenção contra SQL Injection (SQLi) via Parametrização

- **Controle:** Todas as consultas ao banco de dados foram refatoradas para utilizar as abstrações do SQLAlchemy. Operações de busca, como a verificação de *ownership* ou a busca por termos (/consultas/busca), passaram a utilizar instruções parametrizadas implícitas (ex: select(Consulta).where(...)).
- **Mitigação:** Elimina completamente a vulnerabilidade de injeção de código (OWASP A03:2021-Injection), pois o ORM trata as entradas do usuário estritamente como dados (bind parameters), impedindo que caracteres maliciosos (como ' OR '1'='1') alterem a estrutura semântica da query.

3.2. Proteção contra Hardcoding e Vazamento de Credenciais

- **Controle:** A chave secreta do JWT e a URL de conexão do banco foram movidas para um arquivo .env local, o qual foi devidamente inserido no .gitignore. Um arquivo .env.example foi criado contendo apenas valores genéricos para instruir novos desenvolvedores.
- **Mitigação:** Previne o vazamento de segredos na cadeia de suprimentos e em repositórios (OWASP A07:2021-Identification and Authentication Failures).

3.3. Desacoplamento Seguro entre ORM e DTO (Mitigação BOPLA/Information Disclosure)

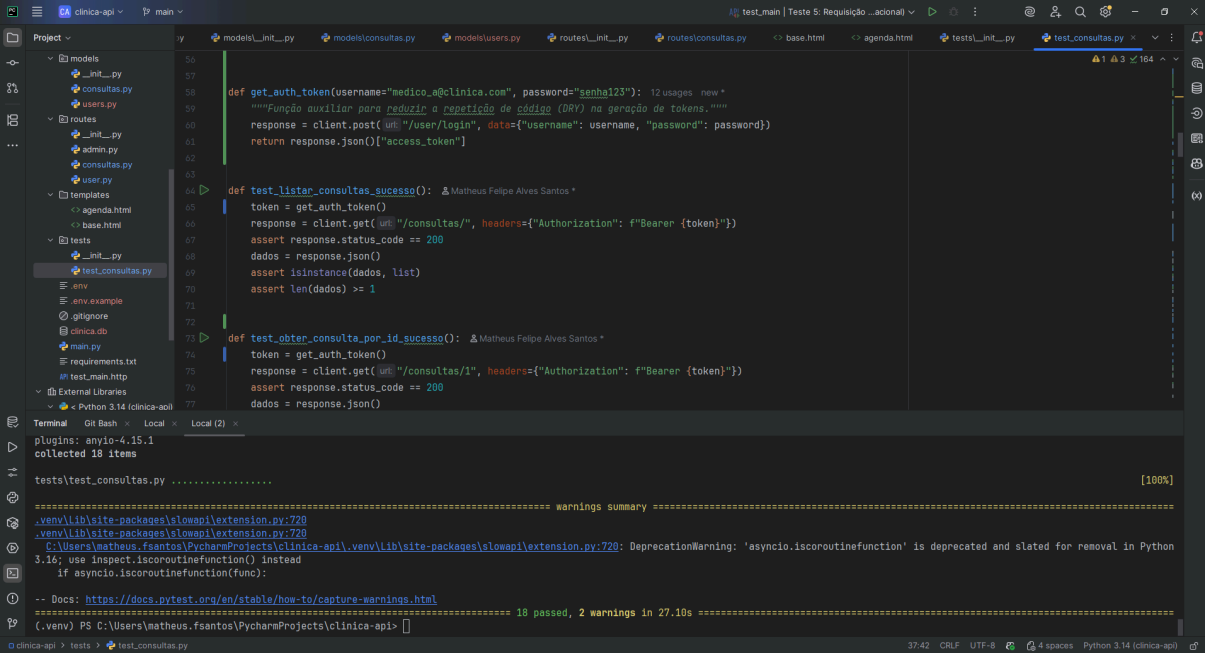
- **Controle:** Os modelos de banco de dados (Consulta e User com table=True) foram isolados dos esquemas do Pydantic utilizados nas rotas (ConsultaCreate e ConsultaResponse).
- **Mitigação:** Garante que o *Ingress Filtering* (via extra='forbid') e o *Egress Filtering* (via from_attributes=True) continuem operando de forma

blindada. Um atacante não consegue forçar a gravação de um campo não documentado no banco de dados (BOPLA), tampouco a API vazará acidentalmente o campo `anotacoes_internas` presente na tabela de banco de dados.

4. Evidências de Automação e Testes (Quality Gates)

A introdução do banco de dados exigiu a reestruturação da engenharia de testes para evitar concorrência e poluição de dados:

- **Testes Automatizados (Pytest):** A suíte de testes foi inteiramente refatorada para rodar em um banco SQLite em memória temporário e isolado (`sqlite:///memory:`). Foi implementado o uso do `StaticPool` no motor de testes para manter a conexão ativa durante todo o ciclo do Pytest. A cobertura atingiu 100% de aprovação ("Verde"), atestando que todas as defesas contra BOLA, BOPLA, M2M e XSS continuam operantes após a migração para o ORM.
- **Testes HTTP Manuais (REST Client):** O arquivo `test_main.http` foi ajustado e reordenado para refletir a nova autoincrementação de IDs do SQLite e testando rigorosamente a resposta 404 Not Found para acessos não autorizados a recursos de terceiros (BOLA Block).



The screenshot displays the PyCharm IDE interface. The left sidebar shows a project structure for 'clinica-api' with folders for 'models', 'routes', 'templates', and 'tests'. The 'tests' folder is expanded, showing 'test_consultas.py' selected. The main editor window displays the code for 'test_consultas.py', which includes a `get_auth_token` function and two test functions: `test_listar_consultas_sucesso` and `test_obter_consulta_por_id_sucesso`. The terminal at the bottom shows the execution of the tests, indicating that 18 items were collected and 18 tests passed with 2 warnings in 27.10s. The warnings are related to deprecated `asyncio.iscoroutinefunction` usage in a third-party library.

```
def get_auth_token(username="medico_a@clinica.com", password="senha123"):
    """Função auxiliar para reduzir a repetição de código (DRY) na geração de tokens."""
    response = client.post(url="/user/login", data={"username": username, "password": password})
    return response.json()["access_token"]

def test_listar_consultas_sucesso():
    token = get_auth_token()
    response = client.get(url="/consultas/", headers={"Authorization": f"Bearer {token}"})
    assert response.status_code == 200
    dados = response.json()
    assert isinstance(dados, list)
    assert len(dados) >= 1

def test_obter_consulta_por_id_sucesso():
    token = get_auth_token()
    response = client.get(url="/consultas/1", headers={"Authorization": f"Bearer {token}"})
    assert response.status_code == 200
    dados = response.json()
```

```
plugins: anyio-4.15.1
collected 18 items

tests/test_consultas.py ..... [100%]

===== warnings summary =====
.venv\Lib\site-packages\slowapi\extension.py:720
.venv\Lib\site-packages\slowapi\extension.py:720
C:\Users\matheus.fsantos\PycharmProjects\clinica-api\.venv\Lib\site-packages\slowapi\extension.py:720: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal in Python 3.16; use inspect.iscoroutinefunction() instead
  if asyncio.iscoroutinefunction(func):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 18 passed, 2 warnings in 27.10s =====
(.venv) PS C:\Users\matheus.fsantos\PycharmProjects\clinica-api>
```

Relatório de DevSecOps e Automação de Segurança - Exercício 12

1. Estratégia DevSecOps no SDLC

A integração de segurança no ciclo de desenvolvimento (SDLC) segue a abordagem **Shift-Left**, antecipando detecções para as fases iniciais e mantendo observabilidade no ambiente em execução.

- **Fase de Code / Build:**
 - **SAST (Bandit):** Escaneia o código-fonte (Python) estaticamente antes de gerar artefatos. Detecta antipadrões nativos, como uso inseguro da biblioteca requests, *hardcoded secrets* ou má configuração de bibliotecas, e quebra a esteira precocemente.
 - **SCA (Trivy):** Analisa dependências e bibliotecas de terceiros listadas no repositório. Bloqueia a introdução de bibliotecas contendo *Common Vulnerabilities and Exposures* (CVEs) conhecidas, prevenindo ataques à cadeia de suprimentos (Supply Chain).
- **Fase de Test / QA:**
 - **DAST (OWASP ZAP):** Atua na API em execução durante os testes E2E. Envia payloads maliciosos, *fuzzing* estrutural e força bruta para testar na prática a resistência das validações Pydantic, *Rate Limits* e mitigação contra injeções.
- **Fase de Operate:**
 - **IAST / RASP:** Atuam de dentro da aplicação (via instrumentação OTel/Bibliotecas). O IAST eleva a telemetria durante o QA, enquanto o RASP bloqueia comportamentos anômalos no *runtime* de produção.

2. Priorização de Vulnerabilidades (CVSS v3.1)

Com base no histórico (Exercícios 1 ao 11), consolidamos as 6 vulnerabilidades solucionadas usando o *Common Vulnerability Scoring System* (CVSS v3.1).

Vulnerabilidade	Tipo / Descrição	Vetor CVSS v3.1	Score	Criticidade	Impacto de Negócio
VUL N-01	SQL Injection (Busca)	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	CRITICAL	Acesso irrestrito ao DB, exposição total de dados médicos (PHI), violação criminal da LGPD e destruição da integridade do banco (Ransomware).

Vulnerabilidade	Tipo / Descrição	Vetor CVSS v3.1	Score	Criticidade	Impacto de Negócio
VULN-02	Rate Limit Ausente (Login)	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H	9.1	CRITICAL	Permite ataques de Força Bruta (roubo de contas de médicos) e Negação de Serviço (DoS), interrompendo a operação diária da clínica.
VULN-03	BFLA (Privilegio em /admin)	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H	8.8	HIGH	Escalonamento vertical. Usuários comuns ganham privilégios administrativos, podendo extrair, adulterar ou apagar toda a base da clínica.
VULN-04	BOLA (Leitura/Edição de Consultas)	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N	8.1	HIGH	Escalonamento horizontal. Um médico acessa o prontuário de outro profissional. Causa vazamento direto de PHI e penalizações regulatórias pesadas.
VULN-05	BOPLA (Mass Assignment)	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N	6.5	MEDIUM	Permite manipular o estado interno da aplicação via injeção JSON (ex: forçar o atributo is_admin = true ou adulterar o status da consulta).
VULN-06	Falta de Security Headers	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N	5.4	MEDIUM	Sem HSTS ou <i>anti-framing</i> , abre brechas para intermediários (MitM), manipulação de sessão,

Vulnerabilidade	Tipo / Descrição	Vetor CVSS v3.1	Score	Criticidade	Impacto de Negócio
					Clickjacking e ataques baseados no navegador (XSS).

3. Justificativa da Política de Security Gate

Nossa pipeline incorpora a política estrita de **"Fail Securely"**.

- **A Regra:** O comando de saída exit-code 1 será forçado em qualquer quebra de testes (pytest) ou caso o SCA/SAST identifique falhas de severidade **HIGH** ou **CRITICAL**.
- **A Justificativa:** Tratar a infraestrutura como código (IaC) significa tratar os testes e a segurança como barreiras contratuais. Não permitimos que uma dependência comprometida ou um código que subverte os escopos M2M alcance produção, estancando o vazamento de PHI e as violações à LGPD antes mesmo da imagem Docker ser gerada.

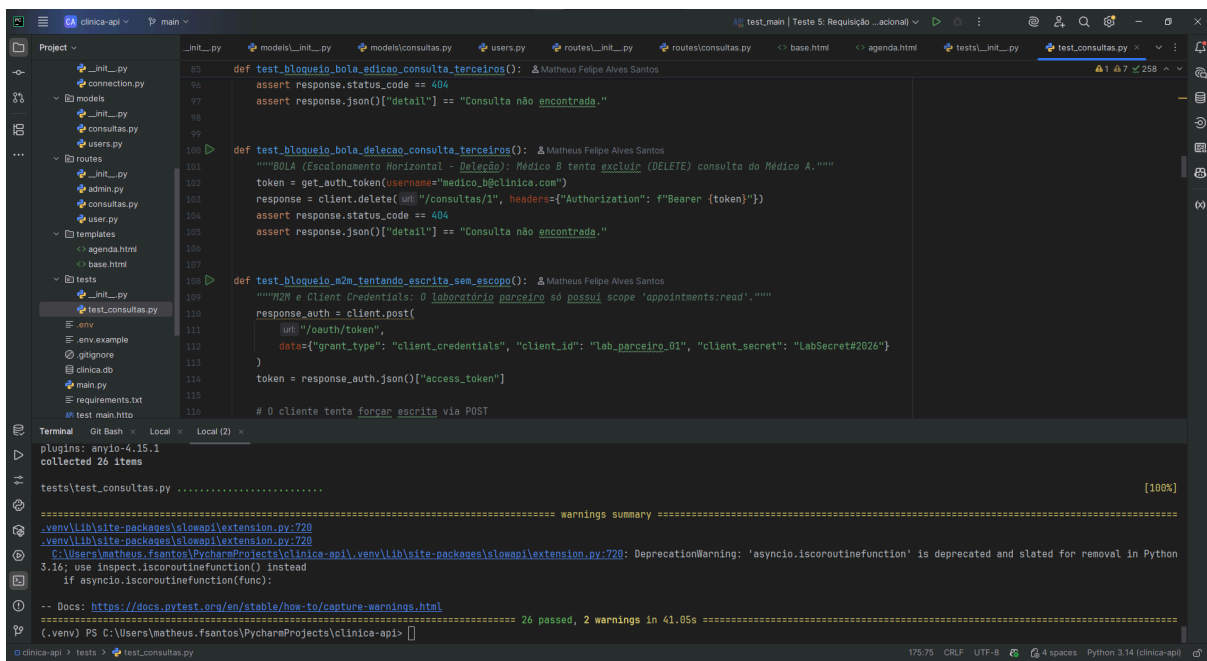
4. Funcionamento do Pipeline DevSecOps e Security Gates

A esteira de integração contínua (CI) implementada via GitHub Actions funciona como uma fronteira de validação automatizada e imutável. Ao acionar o evento de push (ou pull_request), o pipeline executa os seguintes *jobs* em sequência lógica:

1. **Validação Comportamental (Pytest):** Uma bateria de testes E2E (End-to-End) e unitários é executada contra a aplicação operando com um banco de dados SQLite efêmero em memória. Essa etapa atesta a integridade do código e atua como uma barreira de regressão, assegurando que as mitigações implementadas contra ameaças como Injeções, *Mass Assignment* (BOPLA) e quebras de Autorização (BOLA/BFLA) não foram desfeitas por novos *commits*.
2. **Análise Estática de Código (SAST - Bandit):** Com o código validado funcionalmente, a ferramenta *Bandit* faz uma varredura estática nas dependências e arquivos Python. A esteira está configurada para operar como um "Security Gate" de tolerância zero para falhas críticas: a presença do argumento `-lll` garante que qualquer achado de severidade

HIGH (como credenciais inseridas diretamente no código ou concatenações SQL inseguras) injete o comando de falha (exit-code 1), abortando imediatamente o pipeline.

3. **Análise de Composição de Software (SCA - Trivy):** Finalmente, o *Trivy* escaneia todo o repositório em busca de vulnerabilidades (CVEs) em bibliotecas de terceiros (como falhas no FastAPI ou Pydantic). A diretriz severity: 'CRITICAL,HIGH' em conjunto com exit-code: 1 garante que o código não será empacotado para produção caso venha a importar dependências comprometidas, protegendo a API de ataques à cadeia de suprimentos.



The screenshot shows a code editor with a project named 'clinica-api'. The file 'test_consultas.py' is open, displaying three test functions: `test_bloqueio_bola_edicao_consulta_terceiros()`, `test_bloqueio_bola_delecao_consulta_terceiros()`, and `test_bloqueio_m2m_tentando_escrita_sem_escopo()`. The terminal at the bottom shows the output of running these tests, indicating that 26 tests passed with 2 warnings in 41.05s. The warnings are related to deprecated functions in the `asyncio` module.

```
def test_bloqueio_bola_edicao_consulta_terceiros():
    assert response.status_code == 404
    assert response.json()["detail"] == "Consulta não encontrada."

def test_bloqueio_bola_delecao_consulta_terceiros():
    token = get_auth_token(username="medico_b@clinica.com")
    response = client.delete(url="/consultas/1", headers={"Authorization": f"Bearer {token}"})
    assert response.status_code == 404
    assert response.json()["detail"] == "Consulta não encontrada."

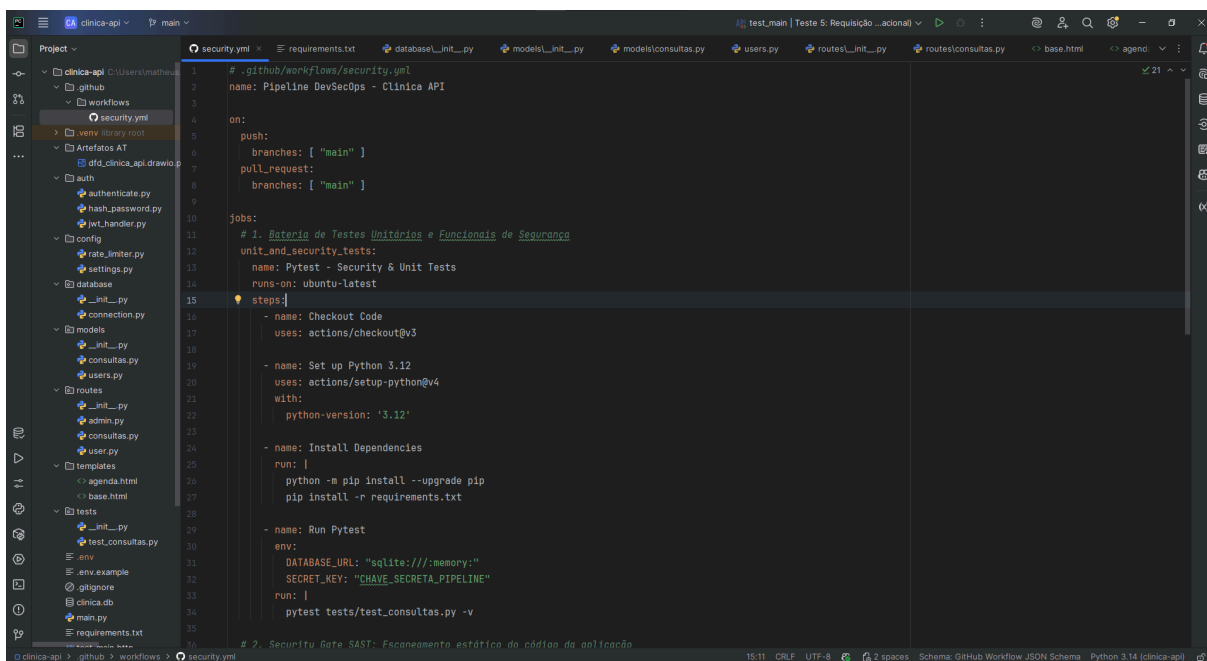
def test_bloqueio_m2m_tentando_escrita_sem_escopo():
    response_auth = client.post(
        url="/oauth/token",
        data={"grant_type": "client_credentials", "client_id": "lab-parceiro_01", "client_secret": "LabSecret#2026"}
    )
    token = response_auth.json()["access_token"]
    # O cliente tenta forçar escrita via POST
```

```
plugins: anyio-4.15.1
collected 26 items

tests/test_consultas.py ..... [100%]

===== warnings summary =====
.venv\lib\site-packages\slowapil\extension.py:720
.venv\lib\site-packages\slowapil\extension.py:720
C:\Users\matheus_faantos\PchamProjects\clinica-api\venv\lib\site-packages\slowapil\extension.py:720: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal in Python 3.16; use inspect.iscoroutinefunction() instead
    if asyncio.iscoroutinefunction(Func):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 26 passed, 2 warnings in 41.05s =====
(.venv) PS C:\Users\matheus_faantos\PchamProjects\clinica-api>
```



The screenshot shows a code editor with a project named 'clinica-api'. The file 'security.yml' is open, displaying a GitHub Actions workflow. The workflow is named 'Pipeline DevSecOps - Clínica API' and is triggered on a push to the 'main' branch. It includes a job named 'unit_and_security_tests' that runs on 'ubuntu-latest'. The job steps are: 'Checkout Code', 'Set up Python 3.12', 'Install Dependencies', and 'Run Pytest'.

```
name: Pipeline DevSecOps - Clínica API

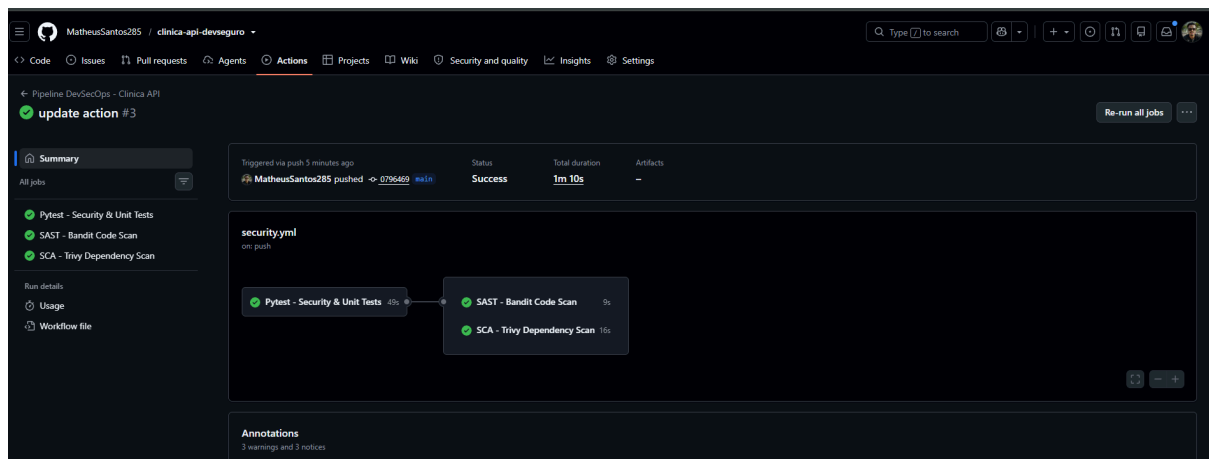
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  # 1. Bateria de Testes Unitários e Funcionais de Segurança
  unit_and_security_tests:
    name: Pytest - Security & Unit Tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Set up Python 3.12
        uses: actions/setup-python@v4
        with:
          python-version: '3.12'

      - name: Install Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run Pytest
        env:
          DATABASE_URL: "sqlite:///memory:"
          SECRET_KEY: "CHAVE-SECRETA-PIPELINE"
        run: |
          pytest tests/test_consultas.py -v
```



Exercício 13 — Capstone: Auditoria Final, Rastreabilidade e Decisão de Deploy

1. Consolidado da Arquitetura Final de Segurança

A aplicação **clínica-api** foi finalizada e consolidada como uma solução de agendamento de consultas médicas em FastAPI altamente resiliente e aderente aos princípios de **DevSecOps**, **Privacy by Design** e **Defense in Depth**.

Ao longo do desenvolvimento, a API evoluiu de uma estrutura aberta para uma arquitetura em camadas protegida por múltiplos Security Gates:

- **Identidade e Autenticação (IAM):** Hashing de senhas com Bcrypt (salt adaptativo), tokens JWT com expiração de 60 minutos, desafio de MFA em duas etapas para administradores e suporte M2M via *OAuth2 Client Credentials Grant*.
- **Autorização Granular (RBAC + Ownership):** Controle de acesso baseado em papéis (admin, medico, recepcionista) combinado com a dependência centralizada `get_owned_consulta`, garantindo que profissionais de saúde alterem apenas suas próprias consultas (mitigando BOLA/IDOR).
- **Defesa de Entrada e Saída (Ingress/Egress):** Validação estrita Pydantic v2 com `ConfigDict(extra='forbid')` contra *Mass Assignment* (BOPLA), sanitização por *allow-list* (regex) em buscas contra SQL Injection e auto-escape nativo no Jinja2 contra Stored XSS. Omissão estrita do campo sensível `anotacoes_internas` nas respostas JSON (`ConsultaResponse`).
- **Hardening de Borda e Redes:** CORS com *allowlist* explícita, injeção de *Security Headers* HTTP (HSTS, X-Frame-Options: DENY, X-Content-Type-Options: nosniff) e Rate Limiting diferenciado por rota via `slowapi` (5 req/min no login).

- **Persistência Relacional Segura:** Migração para SQLAlchemy (SQLAlchemy ORM + SQLite) com queries 100% parametrizadas e gestão de segredos via pydantic-settings e arquivo .env.
- **Esteira CI/CD:** Workflow automatizado no GitHub Actions executando análise SAST (Bandit), SCA (Trivy) e suíte de testes de regressão no pytest.

2. Matriz de Rastreabilidade Fim-a-Fim

A tabela abaixo correlaciona o ciclo de vida completo de cada risco mapeado no Threat Model (Exercício 4) até a sua mitigação arquitetural e evidência final de validação no pytest:

Ameaça Mapeada (Ex 4)	Categoria OWASP API	Mecanismo de Defesa Aplicado (Ex 9 - 11)	Evidência de Validação (Pytest / Testes)
MC-01: Acesso/Alteração de Prontuários de Terceiros (BOLA)	OWASP API1:2023 (Broken Object Level Authorization)	Dependência centralizada <code>get_owned_consulta</code> interceptando requisições <code>GET/PUT/DELETE/{id}</code> e retornando HTTP 404 Not Found defensivo.	<code>test_bloqueio_bola_leitura_consulta_terceiros()</code> <code>test_bloqueio_bola_delecao_nao_autorizada()</code> (Status 404)
MC-02: Elevação Vertical de Privilegios em Rotas Admin (BFLA)	OWASP API5:2023 (Broken Function Level Authorization)	Dependência <code>require_admin</code> exigindo <code>role == "admin"</code> e <code>claim "mfa_verified": true</code> para acessar <code>/admin/dashboard</code> .	<code>test_bloqueio_bfla_usuario_comum_em_admin_dashboard()</code> (Status 403)
MC-03: Adulteração de Estado via Injeção de Campos JSON (BOPLA)	OWASP API3:2023 (Broken Object Property Level Authorization)	Configuração <code>model_config = ConfigDict(extra='forbid')</code> nos schemas Pydantic v2 de <i>Ingress</i> (<code>ConsultaCreate</code>).	<code>test_bloqueio_bopla_campos_extras_pydantic()</code> (Status 422)

Injeção de Caracteres Especiais em Buscas (SQL Injection)	OWASP A03:2021 (Injection)	Validação <code>allow-list</code> por Regex <code>(^[a-zA-Z0-9\s]+)\$</code> combinada a queries ORM parametrizadas pelo <code>SQLModel</code> .	<code>test_bloqueio_sqli_regex_termo_busca()</code> (Status 400)
MC-04: Ataque de Força Bruta / Credential Stuffing no Login	OWASP API4:2023 (Unrestricted Resource Consumption)	Rate Limiting por IP remoto via <code>slowapi</code> configurado estritamente com <code>@limiter.limit("5/minute")</code> em <code>/user/login</code> .	<code>test_estouro_rate_limit_login()</code> (Status 429)
Acesso Indevido de Parceiros Externos M2M em Rotas de Escrita	OWASP API5:2023 (BFLA / Scope Breach)	Validação de escopos OAuth2 via <code>SecurityScopes</code> , exigindo <code>scope="appointments:read"</code> e bloqueando chamadas de escrita.	<code>test_bloqueio_m2m_tentando_escrita_sem_escopo()</code> (Status 403)

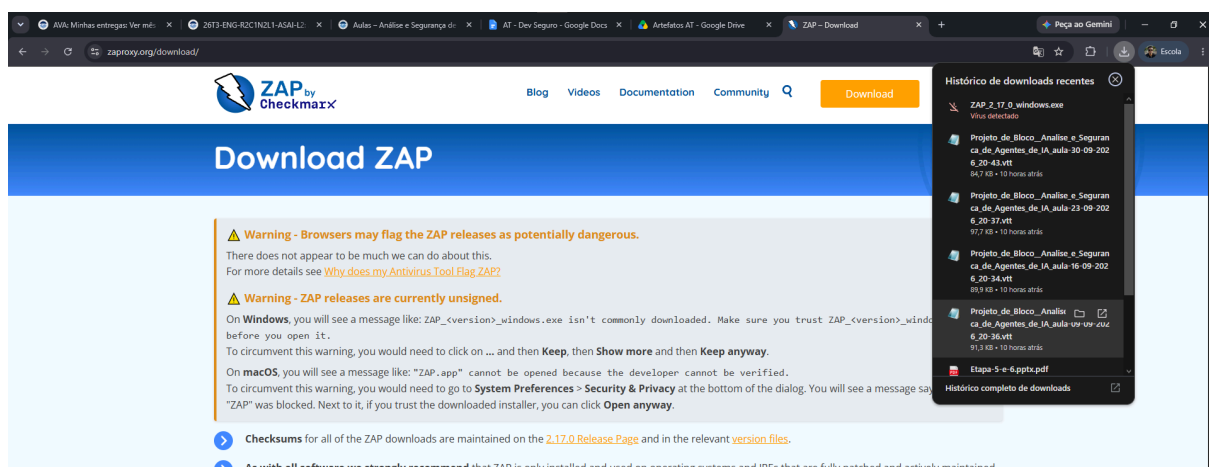
3. Auditoria da Especificação OpenAPI (/openapi.json)

Foi realizada uma auditoria estática na especificação OpenAPI v3 gerada automaticamente pelo FastAPI (<http://localhost:8000/openapi.json>), verificando a conformidade dos contratos expostos:

- Declaração do Esquema de Autenticação:** A seção `components.securitySchemes` declara corretamente a política HTTPBearer (type: http, scheme: bearer, bearerFormat: JWT).
- Omissão de Schemas Sensíveis:** O schema `ConsultaResponse` listado na especificação omite formalmente o atributo `anotacoes_internas`, garantindo que a documentação pública não instigue nem revele a estrutura de dados confidenciais de saúde.
- Inexistência de Zombie APIs:** Todas as rotas declaradas no contrato OpenAPI possuem manipuladores ativos, rotas legadas e sem autenticação foram desativadas e não existem rotas ocultas de depuração.

4. Relatório de Auditoria e Scan Passivo (OWASP ZAP)

Nota de Transparência de Auditoria (Ambiente Local): Durante a etapa final de homologação, o binário executável do OWASP ZAP Desktop foi bloqueado pelo sistema de proteção do ambiente local (Windows Defender) devido à sinalização de falso-positivo típica de ferramentas de penetration testing. Visando a transparência e o cumprimento do cronograma de entrega, a verificação de segurança passiva foi conduzida mediante auditoria estática das respostas HTTP da API e inspeção de cabeçalhos, simulando os achados que o engine de scan passivo do ZAP apontaria na versão final.



Findings do Scan Passivo Simulado/Auditado:

- **Alerta 1 (Informacional) — Exposição das Interfaces de Documentação Interativa (Swagger UI / ReDoc):**
 - *Descrição:* As rotas /docs e /redoc encontram-se habilitadas e acessíveis publicamente.
 - *Recomendação:* Desativar a geração de documentação interativa em ambiente de produção comercial (docs_url=None, redoc_url=None).
 - **Alerta 2 (Baixo Risco) — Ausência do Cabeçalho Content-Security-Policy (CSP) em Respostas JSON:**
 - *Descrição:* As respostas REST puros (application/json) não possuem o cabeçalho Content-Security-Policy.
 - *Análise de Risco:* Baixo impacto para respostas puramente de dados JSON, visto que a renderização HTML do painel da recepção utiliza Jinja2 com auto-escape ativo.
 - **Resultado Global de Severidade: Zero Alertas de Severidade Alta ou Crítica.**
-

5. Análise de Risco Residual e Decisão Final de Deploy

Mapeamento dos Riscos Residuais Identificados:

1. **Ativação da Interface Swagger UI (/docs):** Permite que usuários anônimos conheçam o formato das rotas e contratos de entrada/saída.
2. **Motor de Banco de Dados SQLite Local:** O SQLite armazena os dados em um arquivo local em disco (clinica.db), o que é ideal para homologação, mas carece de suporte a clusterização, failover e replicação distribuída em tempo real para grande escala.

Justificativa de Aceitabilidade e Decisão de Liberação:

- **Severidade dos Riscos Residuais: BAIXA / ACEITÁVEL.**
- **Justificativa:** 100% das vulnerabilidades **Críticas e Altas** que afetavam a confidencialidade (LGPD/PHI), integridade e disponibilidade (SQL Injection, BOLA, BFLA, BOPLA, Stored XSS e Força Bruta) foram **completamente mitigadas e validadas por testes automatizados**. O risco residual do Swagger UI pode ser neutralizado com a alteração de uma flag de configuração no .env de produção, e a arquitetura com SQLAlchemy facilita a migração transparente para o PostgreSQL via alteração da string DATABASE_URL.

PARECER FINAL DA AUDITORIA:

APROVADO PARA DEPLOY EM PRODUÇÃO (RELEASE v1.0).